CHAPTER 4 • CI/CD DELIVERY + MLOPS INTRO ~15 Min

Student

Chapter 3 got AskMyNotes to green CI. What does Chapter 4 add?

Claude • AI Assistant

Chapter 4 extends the automation in two directions: **tighter quality gates BEFORE code leaves your machine**, and **CD + MLOps AFTER code passes CI**. Together, they close the loop from 'my commit compiles' (Chapter 1-2) to 'my model version is auto-promoted to production, with drift monitoring watching for silent regressions' (end of Chapter 4).

This is the transition from 'engineer who ships code' to 'engineer who ships MODEL-BACKED services'. One of Chapter 4's six subsections is high-emphasis (MLflow Tracking, 4.4) because it is the tool you will use on your first day of any ML engineering job.

WHAT CHAPTER 4 COVERS

4.1 Quality Gates. The three-layer defense: **editor plugins** for instant feedback, **pre-commit hooks** for local commit gating, and **CI jobs** for team-wide enforcement. Tools: **ruff** (Rust-fast linter replacing flake8/pylint/isort/pyupgrade/etc.), **black** (opinionated formatter with almost no config), **mypy** (static type checker for verifying your type hints match usage), **pre-commit** (the hook runner that makes them fire on every `git commit`).

One `.pre-commit-config.yaml` file, run in CI via `pre-commit/action@v3`, gives you zero config drift between local and CI.

4.2 CD Basics. Extending CI into Continuous Delivery: the **build-once, deploy-many** principle (same artifact promoted through every environment; never rebuild), the environment ladder (dev -> staging -> canary -> prod), five deployment strategies (**recreate / rolling / blue-green / canary / feature flag**), observability as the safety net (metrics + logs + traces), **SLO-driven auto-rollback**, and GitHub Environments for approval gates. Closes with the pivot: **everything here transfers to MLOps** with 'model + dataset' where 'image' used to be.

4.3 What is MLOps. The definition (CD applied to ML systems - model + code + data + hyperparams as ONE artifact), the three uniquely-ML failure modes (**reproducibility rot, drift, silent failure**) that don't exist in software CD, the seven-category MLOps stack (tracking, registry, data versioning, feature stores, pipeline orchestration, serving, drift monitoring), **Google's MLOps maturity model** (Level 0 manual notebooks -> Level 1

automated pipeline -> Level 2 full CI/CD + drift-triggered retraining), and the two-loop mental model (**dev loop** in minutes-hours, **prod loop** in days-weeks).

4.4 MLflow Experiment Tracking (HIGH-EMPHASIS). The hands-on core of Chapter 4. **MLflow tracking** as Git-for-experiments: experiments and runs, the params/metrics/artifacts trichotomy, **autologging** for standard frameworks (sklearn, transformers, lightning, xgboost, keras), the UI's compare view + parallel-coordinates plots, programmatic queries via `mlflow.search_runs`, reproducibility via git-SHA tags + `pip freeze` artifacts.

AskMyNotes gets a live MLflow server logging every embedding + chunk-size experiment.

4.5 MLflow Model Registry. The bridge from tracked run to served model. Named **registered models** (`AskMyNotesRetriever`), auto-numbered immutable **versions**, mutable **aliases** (`@production`, `@staging`, `@canary` - the modern approach; older fixed-vocabulary **stages** are deprecated), **tags** for governance and approval records, loading `models:/<name>@production` in serving code so promotion is alias-move + rolling restart with zero code change, and the full promotion ladder (candidate -> staging -> shadow -> canary -> production) driven from CI.

4.6 Chapter 4 Review. 35-item vocabulary reference, 10-item self-check, the AskMyNotes upgrade exercise (quality gates + Docker deploys + MLflow tracking + Model Registry with alias-based promotion), and a full Chapter 4 command reference.

THE ASKMYNOTES UPGRADE

Student

What state does AskMyNotes end this chapter in?

Claude • AI Assistant

CONCEPT CARD

By the end of Chapter 4, AskMyNotes has:

- **`.pre-commit-config.yaml`** with ruff + black + mypy + bandit + basic hygiene hooks; installed via `pre-commit install`
- A **`quality` CI job** running `pre-commit/action@v3` on every PR - same rules locally and in CI
- A **`Dockerfile`** and a **`.github/workflows/deploy.yml`** that builds and pushes versioned images to `ghcr.io` on main-branch merges

- **GitHub Environments** for staging (auto-deploy) and production (requires reviewer approval)
- A **local MLflow tracking server** (`mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db`) logging every retrieval experiment
- **~15 tracked experiments** across embedding models, chunk sizes, and overlaps, all comparable in the MLflow UI
- A **registered model `AskMyNotesRetriever`** with multiple versions, tagged with approvals and metrics
- **Alias-based promotion**: `@production` points at the current winner; serving code loads by alias only
- A **`promote_best_model.py`** script that CI can call after each training job to auto-register + promote

That combination is production-grade MLOps for a small team. Any ML platform engineer looking at the repo will read the setup and infer 'this person has shipped ML systems, not just trained models in notebooks'.

REAL WORLD APPLICATION

This exact combination (pre-commit + GitHub Actions + Docker + GHCR + MLflow tracking + Model Registry) is the default MLOps setup at hundreds of small-to-medium companies shipping ML products in 2026. Learning it on AskMyNotes means you can walk into most MLOps interviews and describe the pipeline you've actually built.

That's a far stronger portfolio signal than any certification.

WARNING

Chapter 4 has more moving parts than any earlier chapter - four new tools (ruff, black, mypy, pre-commit) in 4.1 alone, then MLflow tracking + registry across 4.4-4.5. Do the exercises hands-on.

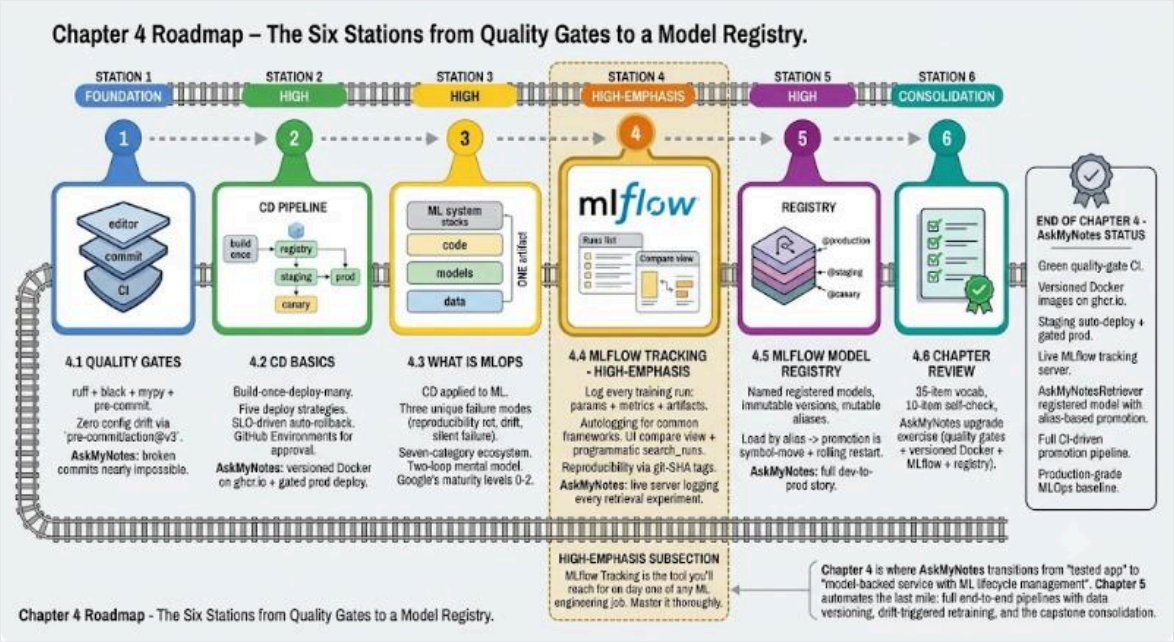
Reading MLflow docs is not a substitute for running `mlflow.start_run` and watching the UI update in real time. AskMyNotes is your practice ground.

EXAM TIP CARD

Interview shortlist for MLOps roles: (1) 'walk me through your quality gate' - answer with the three-layer defense from 4.1; (2) 'how do you promote a model to production?' - answer with the alias-based flow from 4.5; (3) 'how do you know when to retrain?' - answer with drift monitoring from 4.3 + Chapter 5; (4) 'show me an experiment you ran' - open your MLflow UI. The high-emphasis subsection (4.4 MLflow Tracking) is what gives you the portfolio artifact you can actually SHOW.

THE CHAPTER 4 ROADMAP

> Six stations of the Chapter 4 journey - see the roadmap figure below. Each subsection is roughly 2 hours reading + 2 hours hands-on. The high-emphasis subsection (4.4) is longer and denser; budget accordingly.



Chapter 4 Roadmap - The Six Stations from Quality Gates to a Model Registry

Section 4 At-a-Glance - What Each Subsection Covers

Sub	Title	Focus	Emphasis
4.1	Quality Gates	ruff + black + mypy + pre-commit; three-layer defense (editor / commit / CI)	Foundation
4.2	CD Basics	Build-once-deploy-many, environment ladder, five deploy strategies, SLO-driven rollback	High
4.3	What is MLOps?	CD for models + data + code; unique failure modes; seven-category stack; maturity levels	High
4.4	MLflow Tracking	Experiments, runs, params/metrics/artifacts, autologging, search_runs, reproducibility tags	HIGH-EMPHASIS
4.5	MLflow Model Registry	Registered models, versions, aliases (@production etc.), tags, load-by-alias serving pattern	High
4.6	Chapter Review	35-item vocab, 10-item self-check, AskMyNotes MLOps upgrade exercise	Consolidation

bash — Section 4 preview - the quality + CD + MLOps toolbox you'll master

```
# ---- Quality gates (4.1) ----
pip install ruff black mypy pre-commit bandit
pre-commit install                          # sets up .git/hooks/pre-commit
ruff check --fix .                          # lint + auto-fix
black --check .                             # format check (CI-friendly)
mypy askmynotes/                            # type check
pre-commit run --all-files                  # run every hook on every file

# ---- Continuous Delivery (4.2) ----
docker build -t ghcr.io/user/askmynotes:sha-$(git rev-parse HEAD) .
docker push ghcr.io/user/askmynotes:sha-$(git rev-parse HEAD)
gh workflow run deploy.yml -f environment=staging       # auto-deploy
gh workflow run deploy.yml -f environment=production    # gated

# ---- MLflow tracking (4.4) ----
pip install mlflow
mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db
# with mlflow.start_run(run_name='...'):
#     mlflow.log_params({...})
#     mlflow.log_metrics({...})
#     mlflow.log_artifact('confusion.png')
#     mlflow.sklearn.autolog()                # for standard frameworks

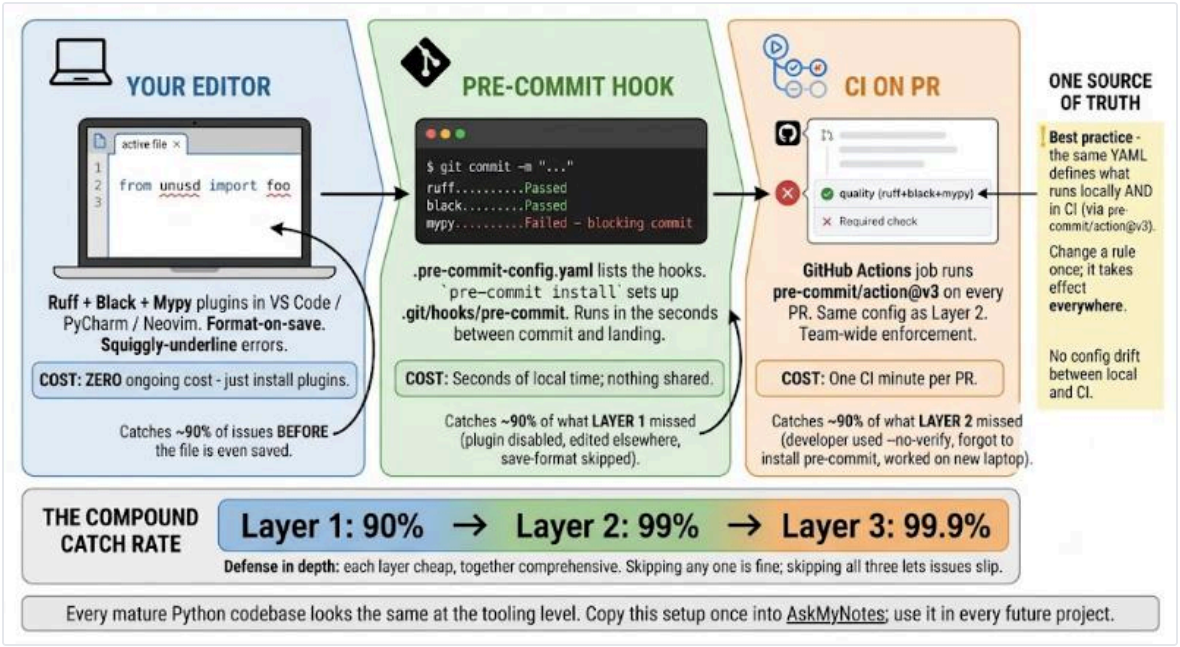
# ---- MLflow Model Registry (4.5) ----
# mlflow.register_model('runs:/<id>/model', 'AskMyNotesRetriever')
# from mlflow import MlflowClient
# MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 8)
# model = mlflow.pyfunc.load_model('models:/AskMyNotesRetriever@production')
mlflow models serve -m models:/AskMyNotesRetriever@production --host 0.0.0.0
--port 8080

# ---- Files you will create in Chapter 4 ----
#  .pre-commit-config.yaml           quality hooks
#  pyproject.toml                    ruff + black + mypy config
#  Dockerfile                        AskMyNotes container image
#  .github/workflows/deploy.yml      build + push + staging + prod deploy
#  experiments/track_run.py          MLflow tracking for retrieval
experiments
#  experiments/promote_best_model.py CI-driven registry promotion
```

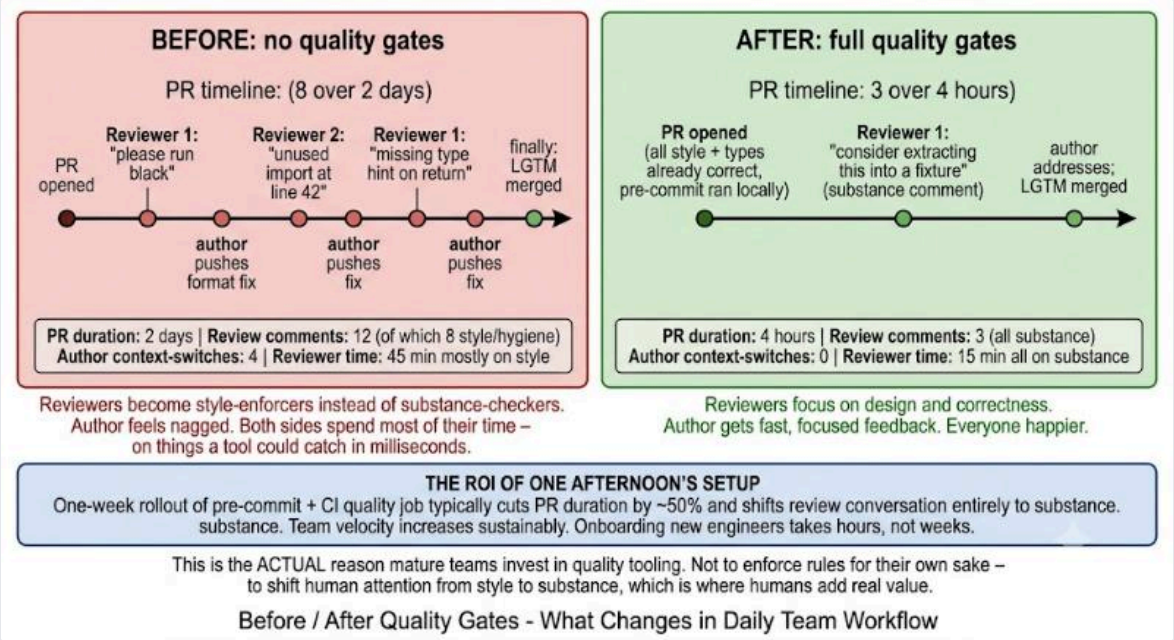
```
#  askmynotes/serving.py                FastAPI loading model by @production
alias
```

QUALITY GATES - RUFF, BLACK, MYPY, PRE-COMMIT

Claude • AI Assistant



The Three-Layer Quality Gate - Editor to Pre-Commit to CI



Before / After Quality Gates - What Changes in Daily Team Workflow

yaml — Complete .pre-commit-config.yaml for AskMyNotes - copy this in

```yaml
# .pre-commit-config.yaml
# After creating this file, run: pre-commit install
# From then on, every `git commit` runs these hooks automatically.

default_language_version:
  python: python3.12

repos:
  # ---- Basic hygiene: trailing whitespace, EOL, YAML syntax, large files --
--
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-toml
      - id: check-json
      - id: check-added-large-files
        args: ['--maxkb=1000']
      - id: check-merge-conflict
      - id: mixed-line-ending
        args: ['--fix=lf']
      - id: debug-statements       # blocks accidental 'breakpoint()' commits
      - id: detect-private-key     # blocks accidental key commits

  # ---- Ruff: lint + auto-fix + format (replaces flake8, isort, pyupgrade,
```

```yaml
...) ----
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.6.9
    hooks:
      - id: ruff
        args: [--fix, --exit-non-zero-on-fix]
      - id: ruff-format

  # ---- mypy: type checking with strict flags ----
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.11.2
    hooks:
      - id: mypy
        # These package stubs are needed for third-party imports
        additional_dependencies:
          - pydantic
          - types-requests
        args: [--config-file=pyproject.toml]

  # ---- Bandit: security-focused lint (hardcoded secrets, weak crypto, etc.)
----
  - repo: https://github.com/PyCQA/bandit
    rev: 1.7.10
    hooks:
      - id: bandit
        args: [-c, pyproject.toml]
        additional_dependencies: ['bandit[toml]']

  # ---- Optional: run tests as a hook (slower - some teams skip this) ----
  # - repo: local
  #   hooks:
  #     - id: pytest-fast
  #       name: pytest (fast tests only)
  #       entry: pytest -x -m 'not slow'
  #       language: system
  #       pass_filenames: false
  #       always_run: true

# ---- Ongoing maintenance ----
# pre-commit autoupdate                 # bump every hook's `rev:` to latest
# pre-commit run --all-files            # run every hook on every file
# pre-commit run ruff --all-files       # run one hook on every file
# git commit --no-verify                # skip hooks (use SPARINGLY - CI will
catch)
```

yaml — The GitHub Actions quality job - one YAML block ties it all together

```yaml
# .github/workflows/ci.yml (add this JOB alongside lint/typecheck/test from Section 3.5)

  # Option A: run pre-commit in CI - guarantees local + CI use the same rules
  quality:
    name: Quality gate (pre-commit)
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
      - name: Install pre-commit
        run: pip install pre-commit
      - name: Cache pre-commit envs
        uses: actions/cache@v4
        with:
          path: ~/.cache/pre-commit
          key: pc-${{ runner.os }}-${{ hashFiles('.pre-commit-config.yaml') }}
      - name: Run all hooks
        run: pre-commit run --all-files --show-diff-on-failure


  # Option B: run tools directly (equivalent, but two sources of truth)
  quality_direct:
    name: Quality gate (direct)
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install -e .[dev]
      - name: Ruff lint
        run: ruff check .
      - name: Ruff format check
        run: ruff format --check .
      - name: mypy type check
        run: mypy askmynotes/
      - name: Bandit security scan
        run: bandit -r askmynotes/ -c pyproject.toml
```

```
# In branch protection settings (Section 3.5 mechanism):
# Require these check names to pass before merge:
#   - Quality gate (pre-commit)
#   - Test (Py 3.12)
#   - Test (Py 3.11)
```

Student

Chapter 3 wired up CI. What does Chapter 4 add?

Claude • AI Assistant

Chapter 3 taught you HOW to run tests and CI. Chapter 4 tightens the loop so **quality checks happen at THREE places** - your editor, your commit, and your PR - each catching the issues the previous layer missed.

This subsection covers the four tools that form the modern Python quality gate (**ruff**, **black**, **mypy**, **pre-commit**), how they compose, and how to wire them into AskMyNotes so a broken commit becomes almost impossible. Chapter 4 then extends the same automation philosophy into Continuous Delivery (4.2) and MLOps (4.3 onward).

WHY LINTERS AND FORMATTERS EXIST

Student

Tests catch bugs. What do linters and formatters catch?

Claude • AI Assistant

CONCEPT CARD

Three categories of issue that tests don't catch:

1. Style inconsistency. Two developers on the same team indent differently, quote strings differently, break lines differently.

Every PR has half its diff hunks about style, not substance. A **formatter** (black) eliminates the debate by applying one canonical style automatically.

2. Latent bugs. Unused imports, unreachable code, `except:` without an exception type, mutable default arguments (`def f(x=[])`), comparing to `None` with `==` . A **linter** (ruff) flags these before they surface as production incidents.

3. Type errors. Passing a `str` where an `int` is expected, calling a method that doesn't exist, forgetting to handle a `None` return. A **type checker** (mypy) uses your type hints to catch these statically. It's the closest Python gets to compile-time safety.

Tests verify behavior; the quality-gate tools verify **hygiene and safety**. Both matter. A codebase with 90% test coverage and zero linting will still be full of subtle bugs and impossible to review.

RUFF - THE LINTER (AND FORMATTER) THAT ATE THE ECOSYSTEM

Student

What is ruff and why does everyone use it?

Claude • AI Assistant

CONCEPT CARD

ruff is a Python linter written in Rust that replaces flake8, pylint, isort, pyupgrade, autoflake, and about a dozen other tools - with rules from all of them combined and runtime measured in **milliseconds even on large codebases**. It's the biggest quality-of-life improvement in Python tooling in years.

bash — Code Example

```bash
pip install ruff

ruff check .                    # lint entire project
ruff check --fix .              # auto-fix what it can
ruff check --watch .            # watch mode - re-lints on file save
ruff format .                   # ruff can also format (black-compatible)
```

Config lives in `pyproject.toml` :

toml — Code Example

```toml
[tool.ruff]
line-length = 100
target-version = 'py312'
```

```
[tool.ruff.lint]
# Enabled rule categories: pycodestyle (E,W), pyflakes (F), isort (I),
# bugbear (B), comprehensions (C4), pyupgrade (UP), simplify (SIM), security
(S)
select = ['E', 'W', 'F', 'I', 'B', 'C4', 'UP', 'SIM', 'S']
ignore = [
    'E501',           # line-too-long (handled by formatter)
    'S101',           # asserts allowed in tests
]

[tool.ruff.lint.per-file-ignores]
'tests/**/*.py' = ['S', 'B011']   # relax security + asserts in tests
```

Ruff's speed matters more than it sounds - because it's fast, you can run it on every save, on every commit, and in CI, without ever feeling the wait. Slow linters get skipped; fast linters get used.

BLACK - THE FORMATTER WITH NO OPTIONS

Student

Isn't formatting subjective?

Claude • AI Assistant

CONCEPT CARD

black decided it isn't. The tool has **almost no configuration** on purpose - the only knob most teams touch is line length. Everything else (quote style, trailing commas, line breaks, indentation) is fixed. This removes bikeshedding entirely.

bash — Code Example

```
pip install black

black .                          # format everything
black --check .                  # dry-run: exit non-zero if anything would
change (CI-friendly)
black --diff .                   # show what would change without changing it
```

Config:

toml — Code Example

```
[tool.black]
line-length = 100
target-version = ['py311', 'py312']
```

ruff-vs-black note: ruff's formatter is now black-compatible and even faster. Many teams have moved to just ruff.

For clarity in learning, we use both here (linter + formatter as separate concerns), but a modern shortcut is `ruff check` + `ruff format` for the whole quality-gate front end.

MYPY - STATIC TYPE CHECKING

Student

I write type hints. What does mypy add?

Claude • AI Assistant

CONCEPT CARD

mypy actually VERIFIES your type hints match how the code is used. Type hints without a checker are documentation - suggestive but unenforced. With mypy, they become gates:

python — Code Example

```python
# askmynotes/chunker.py
def chunk_text(text: str, size: int = 500, overlap: int = 50) -> list[str]:
    if not text:
        return []
    step = size - overlap
    return [text[i:i+size] for i in range(0, len(text), step)]

# Elsewhere:
chunks = chunk_text(pdf_bytes, size=500)     # BUG: passing bytes, not str
```

Running `mypy askmynotes/` catches this before you ever run the code:

code — Code Example

```
askmynotes/pipeline.py:12: error: Argument 1 to "chunk_text" has
incompatible
type "bytes"; expected "str"  [arg-type]
```

Config:

toml — Code Example

```toml
[tool.mypy]
python_version = '3.12'
strict = true                          # all the strict flags at once
(recommended)
warn_unused_ignores = true
warn_return_any = true
show_error_codes = true

# Common relaxations for pragmatic real projects
[[tool.mypy.overrides]]
module = ['tests.*']
disallow_untyped_defs = false          # tests don't need full annotations

[[tool.mypy.overrides]]
module = ['third_party_without_stubs.*']
ignore_missing_imports = true
```

mypy strategy for existing codebases: start non-strict, gradually enable flags module-by-module. Trying to enable `strict = true` on an untyped 50k-line codebase is a two-month project. Add types to new code first; retrofit slowly.

PRE-COMMIT - HOOKS THAT RUN BEFORE CODE LEAVES YOUR MACHINE

Student

How do I make these run automatically?

Claude • AI Assistant

CONCEPT CARD

pre-commit is a tool that installs git hooks running your quality-gate tools automatically on every `git commit`. Miss a formatting step? Pre-commit catches it before the commit even completes.

Install and initialize:

bash — Code Example

```
pip install pre-commit
pre-commit install              # sets up .git/hooks/pre-commit
```

Configure in `.pre-commit-config.yaml` at repo root:

yaml — Code Example

```yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
        args: ['--maxkb=1000']
      - id: check-merge-conflict

  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.6.9
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format

  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.11.2
    hooks:
      - id: mypy
        additional_dependencies: [types-requests, pydantic]
```

Now every `git commit` runs these hooks. If any hook modifies files (ruff --fix, black), the commit ABORTS and you re-stage + re-commit the fixed version. If mypy fails, the commit is blocked entirely.

Manual run:

bash — Code Example

```
pre-commit run --all-files          # run all hooks on all files (great
after adding a new hook)
pre-commit run ruff --all-files     # run one specific hook
pre-commit autoupdate               # bump every hook to its latest version
```

THE THREE-LAYER DEFENSE

Student

Why editor + pre-commit + CI? Isn't that overkill?

Claude • AI Assistant

CONCEPT CARD

Because each layer catches what the previous missed:

Layer 1 - Editor: instant feedback while typing. VS Code, PyCharm, and Neovim all have ruff/black/mypy plugins. Format-on-save + squiggly-underline errors mean most issues never reach even the file save. **Cost**: zero - it's just plugins.

Layer 2 - pre-commit: catches what editor missed (developer disabled plugin, edited in a different editor, forgot to save-format). Runs in the second or two between `git commit` and the commit landing. **Cost**: local seconds; nothing shared or public.

Layer 3 - CI: catches what pre-commit missed (developer skipped hooks via `git commit --no-verify`, or pre-commit isn't installed on that machine). Enforced across the team; nothing merges without passing. **Cost**: a minute in CI time.

Each layer catches ~90% of what would leak through the previous. Three layers = 99.9% catch rate with minimal per-layer cost. This is defense in depth applied to code quality.

WIRING QUALITY GATES INTO GITHUB ACTIONS

Student

Show me the CI part.

Claude • AI Assistant

EXAMPLE CARD

Add a `quality` job to your existing `.github/workflows/ci.yml`:

yaml — Code Example

```yaml
  quality:
    name: Quality gate (ruff + black + mypy)
    runs-on: ubuntu-latest
```

```yaml
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
          cache-dependency-path: 'pyproject.toml'
      - name: Install dev deps
        run: |
          python -m pip install --upgrade pip
          pip install -e .[dev]
      - name: ruff (lint + format check)
        run: |
          ruff check .
          ruff format --check .
      - name: mypy (type check)
        run: mypy askmynotes/
```

Or, even simpler: **run pre-commit in CI** so you have EXACTLY one source of truth (`.pre-commit-config.yaml`) for what's checked. If it passes locally, it passes in CI - because it's literally the same tool config:

yaml — Code Example

```yaml
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
      - uses: pre-commit/action@v3.0.1
```

That's the entire CI quality gate: 6 lines. Config is the pre-commit file. Highly recommended.

ANALOGY

Quality gates are like **airport security layers**: your editor is the friendly check at check-in (obvious issues), pre-commit is the belt-and-scanner (systematic check), CI is the boarding-gate check (final enforcement). Each catches something the others might miss.

Skip one and 90% still works; skip all three and things start slipping through.

REAL WORLD APPLICATION

A team of 12 engineers moved from 'sometimes we run black manually' to a full ruff + black + mypy + pre-commit + CI setup over one week. PR review time dropped from ~2 days to ~4 hours - reviewers no longer left comments about formatting or missing type hints; they focused on logic.

Onboarding new engineers went from days of style adjustment to minutes ('here's the setup script; the tools will guide you'). This is why every mature Python codebase looks the same at the tooling level - the ROI is that good.

WARNING

Do NOT let `# type: ignore` and `# noqa` comments proliferate. Each one is a small tax on the codebase's quality gate.

Require justification comments (`# type: ignore[arg-type] # third-party stub is wrong here - see issue #123`). Grep for these periodically and clean up as libraries improve.

COMMON PITFALL / MISTAKE

Enabling `mypy --strict` on day one of an existing untyped codebase. You'll get thousands of errors and give up.

Instead: enable base mypy first (`mypy .` with default settings), then enable one strict flag per week. `disallow_untyped_defs` -> `warn_return_any` -> `disallow_any_generics` -> `strict_equality` -> full `strict = true` .

Six weeks later you're strict without a big-bang migration.

EXAM TIP CARD

Interview question: 'How do you enforce code quality in a Python team?' Structured answer: 'Three layers of defense: editor plugins for instant feedback (ruff, black, mypy in the IDE), pre-commit hooks locally so the same checks run on git commit (installed via pre-commit install), and CI jobs that run the same tools on every PR - typically via `pre-commit/action@v3` so config lives in one file. Coverage is enforced separately with pytest-cov and Codecov PR comments.

Each layer catches ~90% of what the previous missed.'

PRACTICE QUIZ

Your team has ruff and black in CI but not as pre-commit hooks. What's the likely daily-experience problem?

A) Nothing - CI catches it eventually

B) Developers push code, wait 3 minutes for CI, get a red X on formatting, push a fix, wait another 3 minutes

C) Ruff and black are unnecessary if CI runs them

D) Pre-commit is slower than CI

▼ Reveal Correct Answer

Answer: B. Without pre-commit, every formatting mistake becomes a CI round-trip. Pre-commit catches it in the 2 seconds between typing `git commit` and the commit landing. The layers exist to shift feedback left.

EXERCISE

On AskMyNotes: (a) add ruff, black, mypy, pre-commit to your dev dependencies, (b) create `pyproject.toml` sections for each tool, (c) create `.pre-commit-config.yaml` with the hooks shown, (d) run `pre-commit install` and then `pre-commit run --all-files` - fix everything it flags, (e) add a `quality` job to your CI workflow using `pre-commit/action@v3.0.1`, (f) enable the `quality` check in branch protection alongside `test`.

AI DEEP DIVE PROMPT

Ask an AI: '*Review my pyproject.toml and .pre-commit-config.yaml files. Suggest missing ruff rules I should enable for a production Python service. Recommend a mypy migration strategy for gradually reaching strict = true. Point out any tool overlaps or redundancies.*'

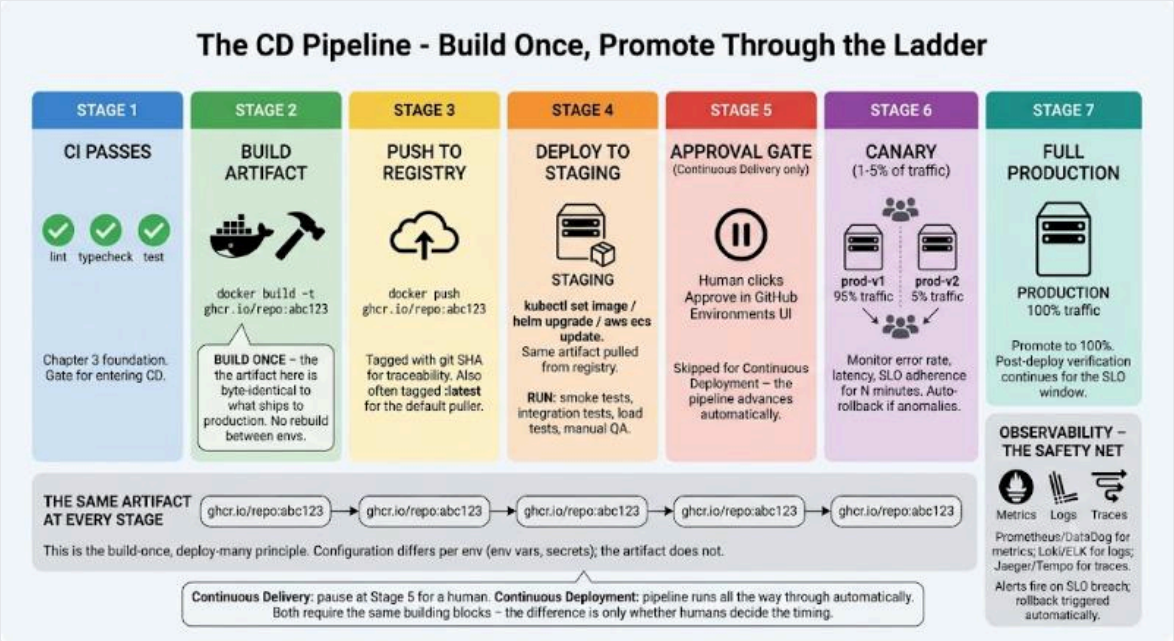
SUMMARY CARD

The Python quality gate: **ruff** (linter, Rust-fast, replaces many old tools), **black** (formatter, no debate), **mypy** (type checker - hints without a checker are documentation), and **pre-commit** (the hook runner that makes them fire automatically on every `git commit`). Wire them into THREE places: **editor plugins** (instant feedback), **pre-commit hooks** (local commit gate), and **CI** (team-wide enforcement, ideally via `pre-commit/action@v3` for one source of truth).

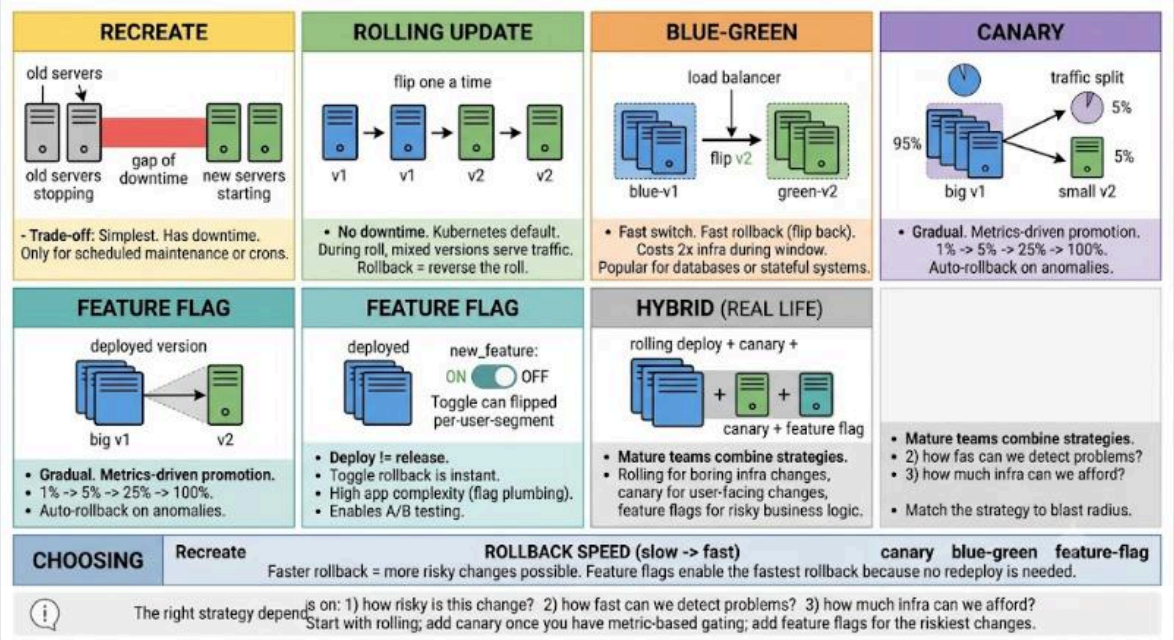
Each layer catches ~90% of what leaks through the previous. Result: broken commits become almost impossible; PR reviews focus on logic instead of style; onboarding is painless.

CD BASICS - FROM BUILD TO DEPLOY

Claude · AI Assistant



The CD Pipeline - Build Once, Promote Through the Ladder



Deployment Strategies Side-by-Side - Blast Radius vs Rollback Speed

yaml — Complete deploy.yml with build-once + staging + gated production for AskMyNotes

```yaml
# .github/workflows/deploy.yml
# Triggered after CI passes on main. Builds Docker image once and
# promotes it to staging automatically, then to production on approval.

name: Deploy
```

```yaml
on:
  workflow_run:
    workflows: ['CI']
    types: [completed]
    branches: [main]
  # Manual trigger for hotfix deploys
  workflow_dispatch:
    inputs:
      environment:
        description: 'Target environment'
        required: true
        default: 'staging'
        type: choice
        options: ['staging', 'production']

concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false     # don't interrupt an in-flight deploy

jobs:
  # ---- Build the artifact ONCE ----
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    if: github.event.workflow_run.conclusion == 'success' ||
github.event_name == 'workflow_dispatch'
    outputs:
      image: ${{ steps.meta.outputs.tags }}
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Extract metadata (tags, labels)
        id: meta
```

```
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/${{ github.repository }}
          tags: |
            type=sha,prefix=sha-
            type=raw,value=latest,enable={{is_default_branch}}

      - name: Build and push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  # ---- Deploy to STAGING - automatic after successful build ----
  deploy_staging:
    needs: [build]
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.askmynotes.example.com
    steps:
      - name: Deploy
        run: |
          echo "Deploying image ${{ needs.build.outputs.image }} to STAGING"
          # Real deploy step goes here. Examples:
          # kubectl set image deployment/askmynotes app=${{
needs.build.outputs.image }}
          # helm upgrade askmynotes ./chart --set image.tag=sha-${{
github.sha }}
          # aws ecs update-service --cluster staging --service askmynotes --
force-new-deployment

      - name: Smoke test
        run: |
          for i in 1 2 3 4 5; do
            curl --fail --max-time 10
https://staging.askmynotes.example.com/health && exit 0
            echo "Health check failed, retry $i/5" && sleep 10
          done
          exit 1

  # ---- Deploy to PRODUCTION - requires manual approval via GitHub
```

```
Environments ----
  deploy_production:
    needs: [deploy_staging]
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://askmynotes.example.com
      # Configured in GitHub UI:
      #   Required reviewers: <team-leads>
      #   Wait timer: 5 minutes (optional cooling-off period)
    steps:
      - name: Deploy (canary rollout)
        run: |
          echo "Rolling image ${{ needs.build.outputs.image }} to PRODUCTION
(canary 5% -> 100%)"
          # Real canary step:
          # kubectl argo rollouts set image askmynotes app=${{
needs.build.outputs.image }}
          # kubectl argo rollouts promote askmynotes

      - name: Verify SLOs
        run: |
          # Check error rate + p95 latency vs pre-deploy baseline for 15 min
          # Auto-rollback triggered by monitoring system if breach detected
          echo "Watching SLOs..."

# GitHub Environments give you: required-reviewer approvals, wait timers,
# per-environment secrets, deployment-history in the UI, and the deploy
# activity feed on the repo homepage.
```

Student

Chapter 3 got CI running. What does CD actually add?

Claude • AI Assistant

CD extends the automation past 'tests passed' to 'the tested artifact is packaged, deployed, and observed'. This subsection covers the CD pipeline shape (build artifact -> push to registry -> deploy to staging -> approval -> deploy to prod), the environments a change traverses, the deployment strategies (rolling, blue-green, canary, feature flag), and the observability required to make automated deploys safe.

This is the last non-ML subsection before we pivot to MLOps in 4.3 - and the mental model here transfers directly: MLOps just adds 'models' and 'datasets' to the list of artifacts a pipeline promotes.

THE CD PIPELINE SHAPE

Student

What does a CD pipeline look like?

Claude • AI Assistant

CONCEPT CARD

A CD pipeline extends the CI pipeline (Section 3.4) with **build + deploy stages**:

1. **CI stages** (from Section 3.5): checkout -> lint -> typecheck -> test.
2. **Build artifact**: build a Docker image (`docker build`), Python wheel (`pip wheel`), or platform-specific bundle. **Build once**; the SAME artifact is promoted through all later environments - never rebuild per environment.
3. **Push to registry**: upload the artifact to a Docker registry (Docker Hub, GitHub Container Registry, AWS ECR, Google Artifact Registry) or PyPI. Tag it with the git SHA for traceability.
4. **Deploy to staging**: pull the artifact from the registry into the staging environment. Run smoke tests + integration tests against the deployed version.
5. **Approval (if Continuous Delivery)**: pause and wait for a human to click 'promote to production' in the Actions UI or Slack. Skipped for Continuous Deployment.
6. **Deploy to production**: pull the same artifact into the production environment. Apply a deployment strategy (rolling / canary / blue-green).
7. **Post-deploy verification**: check metrics (error rate, latency, request volume) for N minutes. Automatically roll back if anomalies detected.

The key discipline: **the artifact promoted to production is byte-identical to the one tested in staging**. Rebuilding for prod is how 'works in staging, broken in prod' happens.

ENVIRONMENTS - THE PROMOTION LADDER

Student

What are dev / staging / prod?

Claude · AI Assistant

CONCEPT CARD

A **promotion ladder** of environments, each closer to real users than the last:

dev: developer laptops. Broken freely; nobody sees except the author.

ephemeral / preview (optional but valuable): a temporary environment spun up for each PR. Reviewers open a URL, click around, verify UI changes. Torn down when PR merges. Common patterns: Vercel/Netlify for frontends, Kubernetes namespaces with unique URLs for backends.

staging: a production-like environment shared by the team. Same infra layout as prod, but with test data and no real users. E2E tests, load tests, and manual QA run here. Environment separates 'code that passes unit tests' from 'code that passes real-app tests'.

canary (optional): a small slice of production - literally 1-5% of real user traffic on the new version. Monitored intensely for anomalies before ramping to 100%.

production: all users. The version everyone runs.

Each rung has its own **gate** to promotion: unit tests -> staging, smoke tests -> canary, healthy metrics -> production 100%. Failure at any gate stops promotion and (usually) triggers rollback.

DEPLOYMENT STRATEGIES

Student

How do teams actually deploy a new version to users?

Claude · AI Assistant

CONCEPT CARD

Five common strategies, in rough order of sophistication:

Recreate (all-at-once): stop the old version, start the new. Simple. Causes downtime. Used only for cron jobs or systems that can tolerate a maintenance window.

Rolling update: replace instances one at a time. No downtime; some users see v1 while others see v2 during the roll. Kubernetes default. Rollback is stopping the roll and reversing it.

Blue-green: run two identical environments (blue = current prod, green = staging with v2). Once v2 is verified, flip the load balancer to send all traffic to green. Rollback is flipping

back to blue - seconds. Costs 2x infra during the swap window.

Canary: gradually shift traffic. Start with 1% of users on v2; observe metrics; promote to 5%, 25%, 100%. Automatic rollback if error rate spikes. Requires strong monitoring and a traffic-splitting layer (Istio, Linkerd, or cloud load balancer features).

Feature flag: v2 is deployed but the new behavior is hidden behind an on/off toggle in code (via LaunchDarkly, Unleash, or a custom flag service). Turn the flag on for internal users first, then 1% of external users, then all.

Rollback is toggling the flag off - instant, no redeploy. Adds application complexity but decouples deploy from release.

In practice, teams combine strategies: rolling deploys for boring infrastructure changes, canary for user-facing behavior changes, feature flags for risky business logic. The right choice depends on blast radius and observability.

BUILD ONCE, DEPLOY MANY

Student

Why 'build once'?

Claude • AI Assistant

CONCEPT CARD

The **build once, deploy many** principle: the artifact promoted to production is BYTE-IDENTICAL to the one tested in staging. This means:

- Build one Docker image, tag it with the git SHA (`myapp:abc123`).
- Push it to the registry.
- Deploy `myapp:abc123` to staging. Test.
- Deploy the SAME `myapp:abc123` to production.

Never rebuild for production. Rebuilding introduces subtle differences (base image updates, dependency versions, timing) that break the 'if staging passed, prod is safe' contract. **The whole point of promoting an artifact is that it's the identical thing you tested.**

Configuration that differs per environment (database URLs, feature flags, log levels) lives OUTSIDE the artifact - environment variables, ConfigMaps, secret stores. The image is universal; the config is environment-specific.

OBSERVABILITY - THE SAFETY NET FOR CD

Student

How do teams sleep at night with auto-deploys running?

Claude • AI Assistant

CONCEPT CARD

Because monitoring + automated rollback catches most issues before humans do. The three pillars of observability:

Metrics: numeric time series - request rate, error rate, latency (p50/p95/p99), CPU/memory. Tools: Prometheus + Grafana, DataDog, Cloudwatch. Alerts fire when metrics breach thresholds ('error rate > 1% for 3 min').

Logs: structured text records of events. Tools: Loki, ELK stack, Splunk. Used for post-hoc debugging.

Traces: end-to-end request paths across microservices. Tools: Jaeger, Tempo, Honeycomb. Used to find bottlenecks and correlate errors across services.

For AskMyNotes at a small scale: even simple metrics from Prometheus + Grafana (or a hosted equivalent) give you the 'is the new deploy healthy?' signal. Automated rollback compares post-deploy metrics to a 15-minute baseline before the deploy; if error rate is significantly higher, roll back automatically.

SLO-driven deployment: teams define Service Level Objectives ('99.9% of requests succeed', 'p95 latency < 500ms'). Deploys that violate SLOs auto-rollback. This is how teams like Google Cloud and Netflix ship hundreds of times per day safely.

CD IN GITHUB ACTIONS - A MINIMAL EXAMPLE

Student

Show me a deploy workflow.

Claude • AI Assistant

EXAMPLE CARD

A deploy workflow that builds a Docker image, pushes it to GitHub Container Registry, and (in a stubbed step) deploys to staging:

yaml — Code Example

```yaml
# .github/workflows/deploy.yml
name: Deploy

on:
  workflow_run:
    workflows: ['CI']
    types: [completed]
    branches: [main]

jobs:
  deploy:
    if: github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.askmynotes.example.com
    steps:
      - uses: actions/checkout@v4

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ghcr.io/${{ github.repository }}:${{ github.sha }}
            ghcr.io/${{ github.repository }}:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Deploy to staging
        run: |
          # In real life: kubectl set image, helm upgrade, aws ecs update-
service, etc.
          echo "Deploying ghcr.io/${{ github.repository }}:${{ github.sha }}
to staging"
```

```
      - name: Smoke test
        run: |
          curl --fail --retry 5
https://staging.askmynotes.example.com/health
```

For production, add a second job with `environment: name: production` - GitHub Environments require an approval from a named reviewer before the job runs. That's how you get Continuous Delivery's 'human clicks to promote' pattern in native Actions.

FROM CD TO MLOPS

Student

How does this connect to MLOps?

Claude • AI Assistant

CONCEPT CARD

Directly. **MLOps is CD applied to machine learning artifacts**.

Where software CD promotes a Docker image through dev -> staging -> prod, ML CD promotes a **trained model** and its **dataset** through the same ladder. All the CD ideas (build once, promote through envs, canary rollouts, feature flags, automated rollback based on metrics) transfer - just with 'model' or 'dataset' where 'image' used to be.

What's DIFFERENT about ML (and what 4.3-4.5 dig into):

- **Model artifacts have provenance**: which dataset trained me, which hyperparameters, which model version. Tracked with MLflow.
- **Metrics are model-specific**: accuracy, F1, MAP@K, drift indicators. Not just latency and error rate.
- **Datasets version too**: not just code and models but the data those models learned from.
- **Deployment can involve two things at once**: a code change AND a model swap. Feature flags become critical.

If CD teaches you 'promote a known-good artifact through a ladder of confidence', MLOps teaches you 'do the same for models, and track everything - dataset, code, hyperparams, metrics - because reproducibility for a model is a much thornier problem than for pure code'.

ANALOGY

CD is the **assembly line for software**. CI ensures the parts (commits) pass inspection; CD assembles them into a package (artifact), promotes the package through progressively-more-realistic testing stations (staging -> canary -> prod), and rolls back the entire batch if quality dips.

The build-once principle is why factories don't remachine parts between stations: a part that passed inspection is the SAME part that goes to the customer.

REAL WORLD APPLICATION

A fintech company moved from 'deploy every Wednesday, entire team on-call' to continuous canary deploys ('every merge to main hits 1% of traffic within 30 minutes, ramps to 100% over 2 hours if metrics stay green'). Result: incident recovery time dropped from ~4 hours (rollback of a batched release) to ~10 minutes (auto-rollback of a canary).

Feature-to-user latency dropped from 5 days to 3 hours. The team stopped having 'release day' entirely - releases became invisible.

WARNING

Don't skip staging just because CD makes deploys fast. Canary catches many issues, but not ones that only appear under load or with specific data patterns. A staging environment running realistic load tests catches these before they reach even a canary population.

COMMON PITFALL / MISTAKE

Deploying different artifacts to different environments. 'We tested v2.1.3 in staging but shipped v2.1.4 to production because we noticed a small fix' - and then v2.1.4 has a bug v2.1.3 didn't.

Rule: what you tested is what you ship, always. If a new fix is needed, promote it through the whole ladder again.

EXAM TIP CARD

Interview question: 'How do you deploy safely to production?' Sharp answer: 'Build once, promote through environments (dev -> staging -> canary -> prod). Use a deployment strategy matched to blast radius: rolling for infra changes, canary for user-facing changes, feature flags for risky business logic.

Gate each promotion on automated checks (tests + metrics + SLO adherence) and have automated rollback triggered by monitoring. Never rebuild between environments - the artifact tested in staging is byte-identical to the one shipped to prod.'

PRACTICE QUIZ

Your team rebuilds its Docker image separately for staging and production 'to make sure it's fresh'. What's the risk?

A) None - fresh images are safer

B) Base-image or dependency updates between the two builds can introduce subtle differences, breaking the 'staging passed = prod safe' contract

C) Building twice wastes CI minutes

D) Both B and C

▼ Reveal Correct Answer

Answer: D. Both are true, but B is the safety issue. The 'build once, deploy many' principle exists specifically to prevent the promotion pipeline from being invalidated by nondeterministic build differences.

EXERCISE

Add a deploy job to AskMyNotes' CI workflow that: (a) builds a Docker image tagged with `${{ github.sha }}`, (b) pushes to `ghcr.io/<your-user>/askmynotes`, (c) uses `environment: name: staging` so it appears as a deploy in the GitHub Environments tab, (d) stubs the actual deploy with an `echo` (unless you want to wire up a real staging server). Confirm the workflow shows a green deploy in the Environments tab after each main-branch push.

AI DEEP DIVE PROMPT

Ask an AI: *'Design a CD pipeline for a Python FastAPI microservice using GitHub Actions. Include build, push to GHCR, deploy to staging with smoke tests, and a gated production deploy requiring approval.

Explain each deployment-strategy trade-off (recreate vs rolling vs blue-green vs canary vs feature flag) for a service like this.'*

SUMMARY CARD

CD extends CI: build an artifact, push to a registry, promote through environments (dev -> staging -> canary -> prod). Golden rule: **build once, deploy many** - the same artifact ships to every environment; never rebuild.

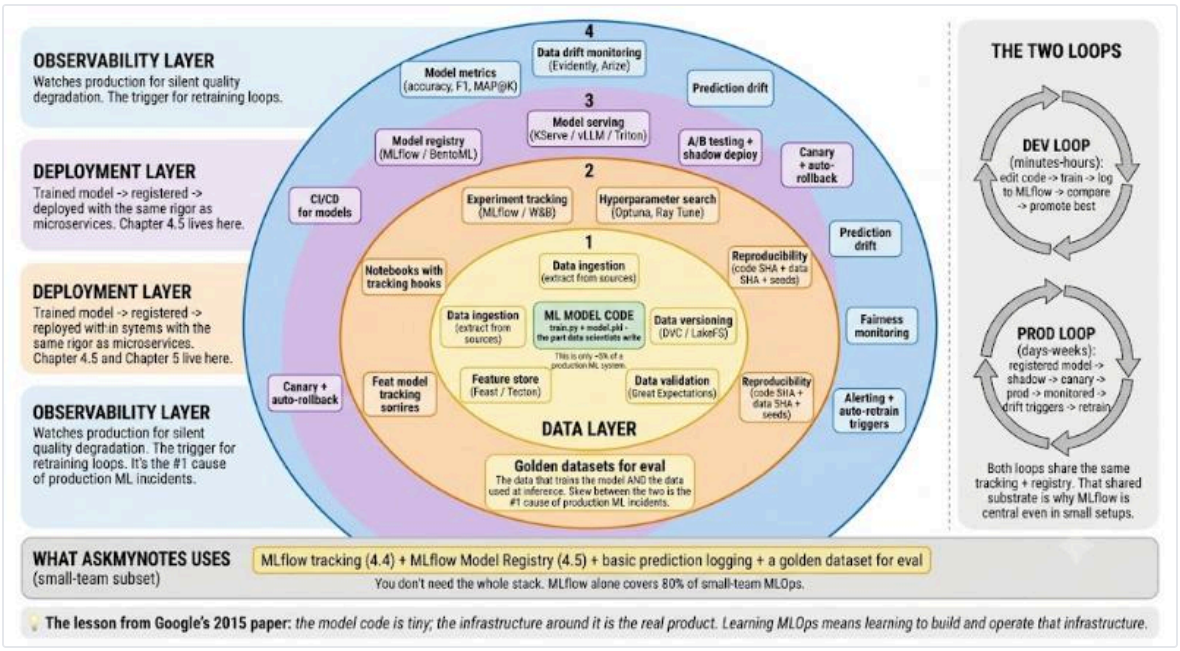
Deployment strategies: recreate (downtime), rolling (default), blue-green (2x infra, fast rollback), canary (small % traffic first), feature flag (deploy != release). **Environments as a promotion ladder** with automated gates at each rung.

Observability (metrics + logs + traces) is what makes automated deploys safe - SLOs drive auto-rollback. Continuous Delivery in GitHub Actions uses **Environments** with required approvals for the prod step.

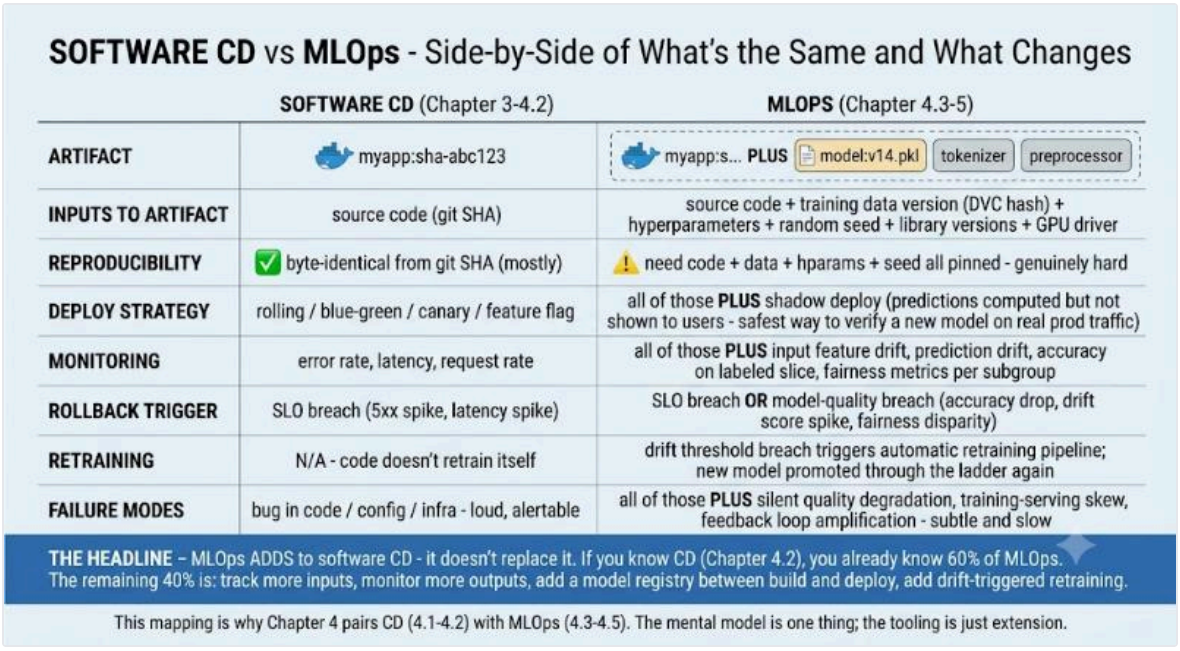
> Everything here transfers to MLOps: replace 'image' with 'model + dataset', add provenance tracking, and you have Chapter 4.3-4.5.

WHAT IS MLOPS - CD APPLIED TO MODELS AND DATA

Claude • AI Assistant



The MLOps System - What Sits Between Model Code and Production Users



Software CD vs MLOps - Side-by-Side of What's the Same and What Changes

bash — The MLOps toolbox - core commands you will see in Chapters 4 and 5

```
# ---- Experiment tracking (4.4 depth) ----
pip install mlflow                              # bundles server + client +
UI
mlflow ui --port 5000                           # local UI at
http://localhost:5000
mlflow ui --backend-store-uri sqlite:///mlflow.db   # persist runs to sqlite

# In Python:
#   import mlflow
#   mlflow.set_experiment('embedding-comparison')
#   with mlflow.start_run():
#       mlflow.log_param('model', 'bge-base')
#       mlflow.log_metric('map_at_5', 0.78)
#       mlflow.log_artifact('confusion_matrix.png')


# ---- Model registry (4.5 depth) ----
# Register a trained model as a named, versioned artifact
#   mlflow.register_model('runs:/<run-id>/model', 'AskMyNotesRetriever')
# Promote a version through stages: None -> Staging -> Production -> Archived
#   client.transition_model_version_stage(name='...', version=3,
stage='Production')


# ---- Data versioning (Chapter 5) ----
pip install dvc
dvc init                                        # creates .dvc/ (like git
init)
dvc add data/raw_pdfs/                          # tracks the folder, stores
hash in .dvc file
git add data/raw_pdfs.dvc .gitignore
git commit -m 'dvc: track raw PDFs (hash in .dvc file)'
dvc remote add -d storage s3://my-bucket/dvc    # or gs://, azure://, local
path
dvc push                                        # upload the actual data
blobs
dvc pull                                        # someone else fetches them


# ---- Pipeline orchestration (Chapter 5 mentions) ----
pip install prefect                             # or: airflow, dagster,
metaflow, zenml
prefect server start                            # UI at http://localhost:4200
```

```
# In Python:
#   from prefect import flow, task
#   @task
#   def train(): ...
#   @flow
#   def training_pipeline():
#       model = train()
#       ...


# ---- Model serving ----
pip install bentoml                             # or fastapi + your own
serving code
bentoml build                                   # bundle model + service into
a container
bentoml serve service.py:svc --production


# ---- Drift monitoring (Chapter 5 mentions) ----
pip install evidently                           # generate drift reports as
HTML dashboards
# from evidently.report import Report
# from evidently.metric_preset import DataDriftPreset
# Report(metrics=[DataDriftPreset()]).run(reference_data=ref,
current_data=prod).save_html('drift.html')
```

Student

Everyone at ML conferences talks about MLOps. What actually is it?

Claude • AI Assistant

MLOps is DevOps + CD applied to machine learning systems. Where DevOps automates 'build, test, deploy, monitor' for code, MLOps does the same for **models + datasets + code together as one system**.

The core insight: an ML model in production is much harder to keep working than a stateless microservice, because its behavior depends on things Git alone doesn't track - training data, hyperparameters, random seeds, feature engineering, upstream data schemas. MLOps is the set of practices that give ML systems the same reliability discipline software engineering has enjoyed for decades.

This subsection covers what MLOps actually contains (not just tracking, but versioning + validation + serving + monitoring + governance), why it's harder than software CD, and the ecosystem you'll encounter (MLflow, DVC, feature stores, model registries, Kubeflow/Metaflow/ZenML). Section 4.4 goes deep on MLflow experiment tracking, and 4.5 on the Model Registry.

Chapter 5 wires them into full pipelines.

WHY ML SYSTEMS NEED SPECIAL OPS DISCIPLINE

Student

Isn't a model just another Python function to deploy?

Claude • AI Assistant

CONCEPT CARD

It is - and it isn't. From an infrastructure view, `model.predict(x)` is just a function. But three things about ML systems make them uniquely fragile:

1. Reproducibility is genuinely hard. Rebuilding a code binary from a git SHA is a solved problem.

Rebuilding a trained model from a git SHA requires the same code + the same training data + the same hyperparameters + the same random seed + often the same GPU driver version. Miss any one and you get a subtly different model.

Without disciplined tracking, 'why is production behaving differently than my notebook?' becomes an unanswerable question.

2. Behavior depends on data that changes. A microservice's behavior depends on its code and config.

A model's behavior depends on its code, config, AND the statistical properties of the data it was trained on. When production data drifts away from training data (a new user demographic, a schema change upstream, seasonal shifts), model accuracy silently degrades - **there's no exception, no error log, just wrong answers**.

This is called **data drift** or **concept drift** depending on the flavor.

3. Failure modes are subtle. A broken microservice returns HTTP 500 - loud, obvious, alertable.

A broken model returns confidently wrong predictions - silent, requires downstream metric monitoring to detect. You can't just check 'is the service up?'; you need 'is the model still accurate?'.

These three properties are why MLOps exists as a distinct discipline. Any of them alone could be handled by careful engineering; together they demand systematic tooling.

THE MLOPS STACK - WHAT TOOLS COVER WHAT

Student

What tools are in the MLOps ecosystem?

Claude • AI Assistant

CONCEPT CARD

Roughly seven categories, each with several popular tools:

Experiment tracking: log every training run's inputs (code SHA, dataset version, hyperparams) and outputs (metrics, artifacts). **MLflow** (open source, most popular), **Weights & Biases** (SaaS, richer UI), **Neptune.ai**, **Comet**, **ClearML**.

Chapter 4.4 covers MLflow depth.

Model registry: named, versioned store of trained models with promotion stages (Staging -> Production). **MLflow Model Registry** (Section 4.5), **BentoML**, **AWS SageMaker Model Registry**.

Data versioning: Git-like versioning for datasets too big for Git. **DVC** (Git-adjacent), **LakeFS**, **Pachyderm**, **lakeFS**, **Delta Lake** (Databricks). Section 5.1 goes deeper.

Feature stores: centralized feature computation + serving; the same feature values used at training and at inference. **Feast** (open source), **Tecton**, **AWS SageMaker Feature Store**, **Databricks Feature Store**.

Pipeline orchestration: schedule + monitor multi-step ML workflows (extract -> preprocess -> train -> validate -> register -> deploy). **Airflow**, **Prefect**, **Dagster**, **Kubeflow**, **Metaflow**, **ZenML**.

Model serving: turn a trained model into a REST/gRPC endpoint. **BentoML**, **Seldon**, **KServe** (formerly KFServing), **NVIDIA Triton**, **vLLM** (for LLMs), **TorchServe**, **TensorFlow Serving**.

Monitoring & drift detection: watch model inputs and outputs in production. **Evidently AI** (open source), **Arize**, **WhyLabs**, **Fiddler**, **Aporia**.

End-to-end platforms (batteries included, opinionated): **Databricks**, **AWS SageMaker**, **Google Vertex AI**, **Azure Machine Learning**. These bundle most of the above under one UI.

For AskMyNotes-scale learning, you don't need all of these - **MLflow alone covers tracking + registry + serving** at a level sufficient for small production ML apps. Chapter 4.4-4.5 does exactly that.

MLOPS MATURITY LEVELS

Student

How do teams progress in MLOps?

Claude • AI Assistant

CONCEPT CARD

The Google Cloud MLOps maturity model (widely cited) defines three levels:

Level 0 - Manual ML process. Notebooks are the artifact. Models trained by hand, deployed by hand, updated when someone remembers. No CI/CD for ML. Common at very early stages or in research contexts.

Level 1 - ML pipeline automation. The training pipeline is automated and versioned. Trigger a new training run programmatically; artifacts land in a registry.

Deployments are still manual, but training and evaluation are code, not notebooks. This is where most production ML teams live.

Level 2 - Full CI/CD for ML. Code changes trigger the training pipeline; model quality is auto-validated; approved models are auto-deployed. Data drift and prediction drift trigger retraining automatically.

Feature store keeps training/serving skew away. This is what large ML platforms (Netflix, Uber, Meta) run.

The maturity ladder mirrors software CD's ladder from 'deploy on Wednesday afternoons' to 'multi-per-day auto-deploys'. Chapter 4 and 5 walk you from Level 0 to a solid Level 1 for AskMyNotes.

ASKMYNOTES AND MLOPS - THE CONCRETE STORY

Student

Where does AskMyNotes need MLOps?

Claude • AI Assistant

CONCEPT CARD

AskMyNotes has multiple ML components, each with reproducibility + drift concerns:

1. Embedding model (turns text chunks into vectors). Choosing between `all-MiniLM-L6-v2`, `bge-base-en-v1.5`, `text-embedding-3-small` is an experiment - tracked with MLflow.

2. Chunking strategy (size, overlap, semantic vs character). Also an experiment - many parameter combos, each measured on retrieval quality (MAP@5, MRR).

3. Retrieval reranker (optional cross-encoder that re-scores initial results). Its choice + parameters are yet another experiment.

4. LLM prompt template for the answer generation step. Different prompts produce different answer quality - tracked as an experiment.

5. Full-pipeline evaluation on a fixed set of Q-and-A pairs (a **golden dataset**). Every code change to any component is measured against this benchmark. Regressions block deploys.

Without MLflow: 'we tried a bunch of embedding models three months ago and picked one, don't remember the numbers'. With MLflow: 'the run comparison shows bge-base beats MiniLM by 8 points on MAP@5 but is 3x slower; here's the tradeoff plot; we picked bge-base with a smaller top-K to compensate'.

That's real engineering vs. vibes.

THE TWO LOOPS OF AN MLOPS SYSTEM

Student

Is MLOps just one big pipeline?

Claude • AI Assistant

CONCEPT CARD

Two interlocking loops:

The DEV loop (fast, run by ML engineers): edit code / config -> run training locally or on a training cluster -> log metrics to MLflow -> compare runs -> promote the best model to the registry. Iteration time: minutes to hours.

The PROD loop (slow, mostly automated): a model version in the registry is promoted to Staging -> deployed to a shadow environment (predictions computed but not shown to users) -> if metrics are good, promoted to canary -> if metrics still good, promoted to

production. Monitoring watches for drift; when drift crosses a threshold, retraining is triggered automatically, restarting the dev loop.

Section 4.4 wires up the dev loop for AskMyNotes with MLflow tracking. Section 4.5 adds the model registry as the bridge to the prod loop. Chapter 5 completes the prod loop with pipeline automation and drift-triggered retraining.

THE SEVEN BIG MLOPS FAILURE MODES

Student

What actually goes wrong in production ML?

Claude • AI Assistant

⚠ **WARNING**

Seven failure modes that MLOps practices exist to catch:

1. **Training-serving skew**: features computed differently at training time and inference time. The classic culprit - a Pandas one-liner in the training notebook, a hand-rolled Python function in the serving code, subtly different results.

1. **Data drift**: input distribution shifts (new user demographic, feature schema change upstream, sensor drift). Model was trained on old data; predictions on new data degrade silently.

1. **Concept drift**: the relationship between inputs and outputs changes. Same inputs used to predict class A now predict class B (customer behavior shifts, adversarial actors adapt).

1. **Silent failure**: model returns a plausible-looking wrong answer with no error. No 500, no traceback, just bad predictions until a human notices.

1. **Broken retraining pipeline**: pipeline runs, model retrains, but on stale or corrupted data. New 'improved' model is actually worse. Auto-deploy without validation causes an outage.

1. **Reproducibility rot**: three months later, someone tries to reproduce a training run for debugging. Original data is gone, hyperparams weren't logged, library versions changed. Nobody can rebuild the model that's currently in prod.

1. **Feedback loops**: model predictions influence future training data (recommender systems especially). Model amplifies its own biases over time; drift is caused by the model itself.

MLflow + a model registry + drift monitoring + a feature store together prevent or detect all seven. Chapter 4 focuses on #6 (tracking / reproducibility) and #5 (validated model promotion). Chapter 5 covers #2/#3/#4 (monitoring + drift).

ANALOGY

MLOps is **the operating system for ML products**. Just as a real OS handles memory, scheduling, and I/O so applications don't have to, MLOps handles experiment tracking, artifact versioning, deployment orchestration, and monitoring so data scientists can focus on modeling.

Without an OS, every application would reimplement filesystem code; without MLOps, every ML team reimplements experiment tracking (usually badly, in spreadsheets).

REAL WORLD APPLICATION

Google's 2015 paper 'Hidden Technical Debt in Machine Learning Systems' catalogued the operational nightmares of running ML in production - and became the founding text of MLOps. Its central observation: in a real ML system, the **ML code is a tiny fraction** of the surrounding infrastructure (config, data collection, verification, feature engineering, process management, monitoring, serving).

MLOps is the recognition that the surrounding infrastructure is the actual product; the model is just one component of it.

WARNING

Don't buy the entire MLOps ecosystem on day one. Teams commonly waste six months evaluating and integrating Kubeflow + Feast + Seldon + Evidently + DVC + MLflow, then realize they had 2 models in production and could have gotten by with MLflow + a `cron` job.

**Start with MLflow tracking (4.4). Add the registry (4.5) when you deploy models.

Add drift monitoring when you have production traffic. Add pipeline orchestration when you have multiple models retrained on different cadences.

Adopt tools in response to actual pain, not to complete a checklist.**

COMMON PITFALL / MISTAKE

Treating models as code artifacts and stopping there. Git alone does not version the data that trained the model, the hyperparameters, or the metrics.

A git-tagged model file is not a reproducible experiment - it's a mystery box. MLflow (or an equivalent) is the minimum viable tracking layer above Git.

EXAM TIP CARD

Interview question: 'What's the difference between DevOps and MLOps?' Sharp answer: 'MLOps extends DevOps to cover the ML-specific artifacts that Git alone doesn't version: training data, hyperparameters, model files, evaluation metrics. It adds tooling for experiment tracking (MLflow), model registries with promotion stages, data versioning (DVC), feature stores (Feast), and drift monitoring (Evidently).

The core practices - CI/CD, rollback, observability - are the same, applied to a wider set of artifacts and a broader set of failure modes (data drift, silent quality degradation, training-serving skew).'

PRACTICE QUIZ

Your team ships a well-tested classification model. Two months later, accuracy has silently dropped from 92% to 78% in production despite no code changes. What's the most likely cause?

A) A latent bug in the model code

B) Data drift - the input distribution has shifted since training

C) Kubernetes is misconfigured

D) The model file corrupted on disk

▼ Reveal Correct Answer

Answer: B. Silent accuracy degradation without code changes is the textbook signature of data or concept drift. This is why MLOps adds drift monitoring on top of standard observability - error rate and latency stayed healthy the whole time.

EXERCISE

On AskMyNotes: (a) list every ML component (embedding model, chunker params, LLM prompt, reranker if any), (b) for each, name the inputs (code, hyperparams, data) that would need to be tracked to reproduce a specific run, (c) identify one 'experiment question' worth answering (e.g., 'does bge-base beat MiniLM on our top 20 test questions?'), (d) sketch how you would answer it TODAY without MLflow - notice how quickly it becomes 'I'd manually record numbers in a spreadsheet'. That gap is what MLflow fills in Section 4.4.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain MLOps to a software engineer who knows CI/CD but hasn't shipped ML models. Cover why data + models + code together are the artifact, the seven big failure*

> *modes (training-serving skew, drift, silent failure, retraining pipeline bugs, reproducibility rot, feedback loops), and the minimum viable MLOps stack for a small team.'*

SUMMARY CARD

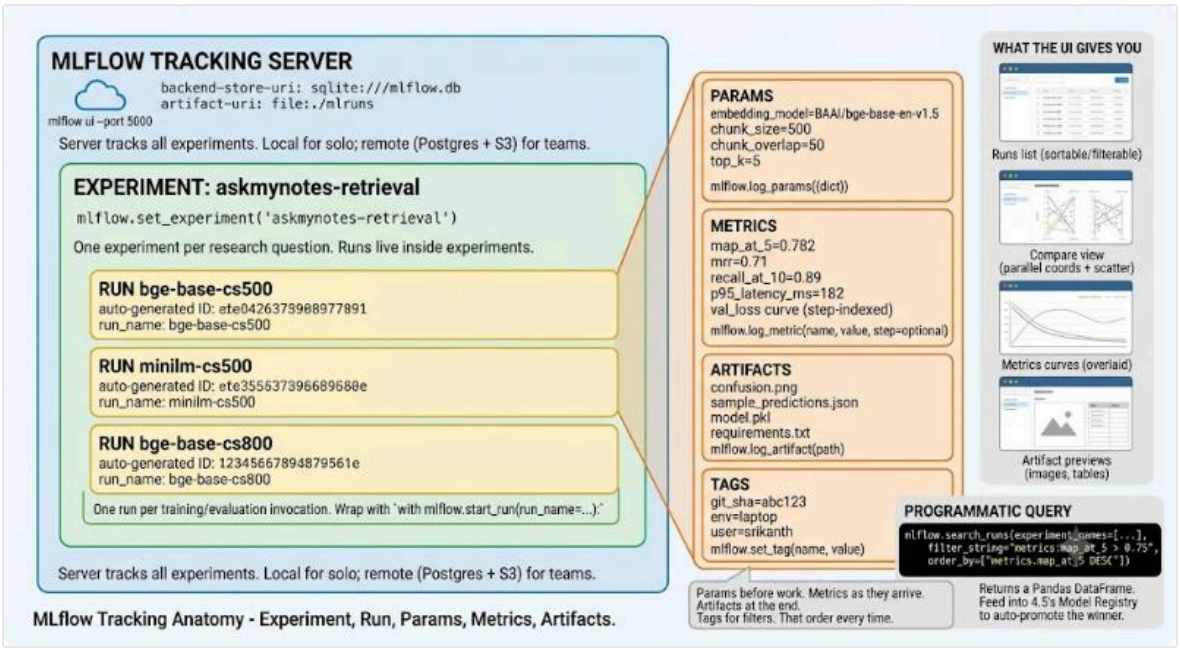
MLOps = DevOps + CD applied to ML systems, where the artifact is code + data + model + hyperparameters, not just code. Extra concerns beyond software CD: **reproducibility rot** (nobody can rebuild a 3-month-old model), **data / concept drift** (silent quality degradation), **training-serving skew** (features computed differently in training vs prod), and **silent failures** (wrong predictions, no exceptions).

The stack: experiment tracking (**MLflow**, W&B), model registries (**MLflow Model Registry**), data versioning (**DVC**), feature stores (**Feast**), pipeline orchestration (**Airflow, Prefect, Metaflow**), model serving (**BentoML, KServe, vLLM**), drift monitoring (**Evidently, Arize**). **Google's maturity model**: Level 0 (manual notebooks) -> Level 1 (automated training pipeline) -> Level 2 (full CI/CD for ML with drift-triggered retraining).

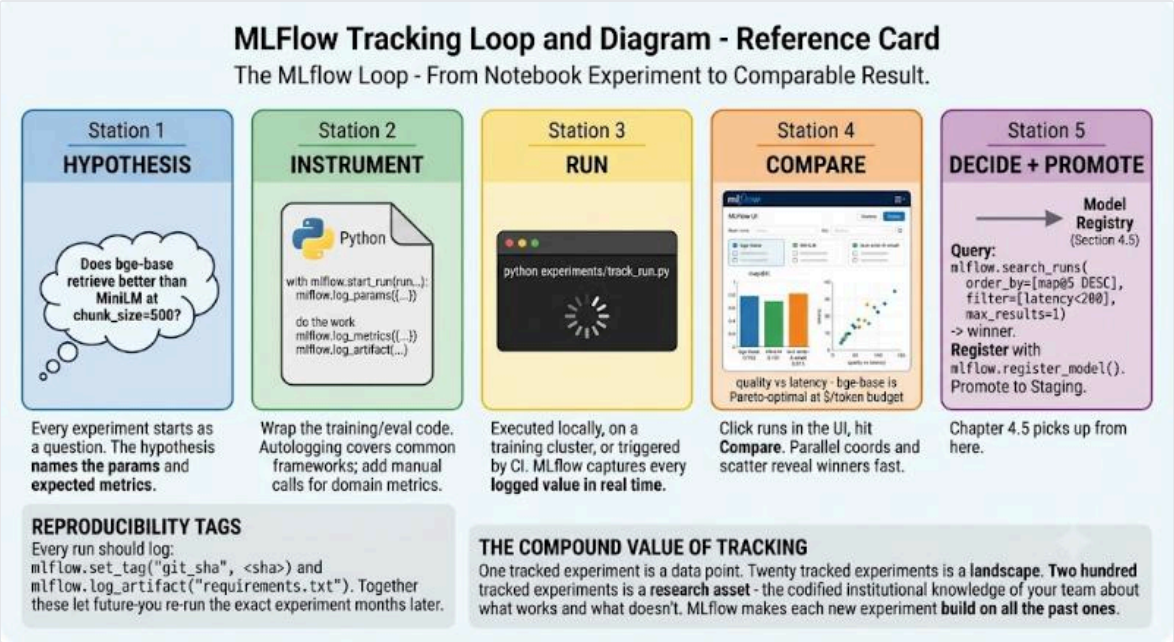
Start with MLflow tracking; add tools in response to actual pain, not to complete a checklist.

MLFLOW EXPERIMENT TRACKING - THE HIGH-EMPHASIS HANDS-ON

Claude • AI Assistant



MLflow Tracking Anatomy - Experiment, Run, Params, Metrics, Artifacts



The MLflow Loop - From Notebook Experiment to Comparable Result

python — Complete tracked-experiment script for AskMyNotes - copy this and adapt

```python
# experiments/track_retrieval_run.py
# Runs one AskMyNotes retrieval experiment and logs everything to MLflow.
# Usage: python experiments/track_retrieval_run.py --model bge-base --chunk-size 500

import argparse
import subprocess
from pathlib import Path

import mlflow
import matplotlib.pyplot as plt

from askmynotes.chunker import chunk_text
from askmynotes.embedder import embed_batch
from askmynotes.retrieval import build_index, top_k_search
from askmynotes.eval import evaluate_retrieval
from askmynotes.data import load_golden_qa


def parse_args():
    p = argparse.ArgumentParser()
    p.add_argument('--model', default='BAAI/bge-base-en-v1.5')
    p.add_argument('--chunk-size', type=int, default=500)
    p.add_argument('--chunk-overlap', type=int, default=50)
    p.add_argument('--top-k', type=int, default=5)
    p.add_argument('--tracking-uri', default='http://localhost:5000')
    p.add_argument('--experiment', default='askmynotes-retrieval')
```

```
    return p.parse_args()


def git_sha() -> str:
    try:
        return subprocess.check_output(['git', 'rev-parse',
'HEAD']).decode().strip()
    except Exception:
        return 'unknown'


def main():
    args = parse_args()

    mlflow.set_tracking_uri(args.tracking_uri)
    mlflow.set_experiment(args.experiment)

    run_name = f"{args.model.split('/')[-1]}-cs{args.chunk_size}-
ov{args.chunk_overlap}"

    with mlflow.start_run(run_name=run_name):
        # ---- 1. Tags for reproducibility ----
        mlflow.set_tag('git_sha', git_sha())
        mlflow.set_tag('user', 'srikanth')
        mlflow.set_tag('env', 'laptop')

        # ---- 2. Params (fixed inputs) ----
        mlflow.log_params({
            'embedding_model': args.model,
            'chunk_size': args.chunk_size,
            'chunk_overlap': args.chunk_overlap,
            'top_k': args.top_k,
        })

        # ---- 3. requirements.txt as an artifact ----
        reqs = subprocess.check_output(['pip', 'freeze']).decode()
        Path('/tmp/requirements.txt').write_text(reqs)
        mlflow.log_artifact('/tmp/requirements.txt')

        # ---- 4. Do the actual work ----
        docs = load_golden_qa()
        all_chunks = []
        for d in docs:
            all_chunks.extend(chunk_text(d.text, size=args.chunk_size,
                                         overlap=args.chunk_overlap))
        embeddings = embed_batch(all_chunks, model=args.model)
```

```
        index = build_index(embeddings)

        results = []
        for q in docs:
            hits = top_k_search(index, q.question, k=args.top_k)
            results.append({'q': q.question, 'hits': hits, 'gold':
q.gold_chunks})

        metrics = evaluate_retrieval(results)

        # ---- 5. Metrics ----
        mlflow.log_metrics({
            'map_at_5': metrics['map@5'],
            'mrr': metrics['mrr'],
            'recall_at_10': metrics['recall@10'],
            'p95_latency_ms': metrics['p95_ms'],
            'total_chunks': len(all_chunks),
        })

        # ---- 6. Artifacts (files) ----
        fig, ax = plt.subplots(figsize=(8, 4))
        ax.bar(['MAP@5', 'MRR', 'Recall@10'],
               [metrics['map@5'], metrics['mrr'], metrics['recall@10']])
        ax.set_ylim(0, 1)
        ax.set_title(f'Retrieval Quality - {run_name}')
        mlflow.log_figure(fig, 'retrieval_quality.png')
        plt.close(fig)

        mlflow.log_dict(
            {'sample_predictions': results[:10]},
            'sample_predictions.json',
        )

        print(f"Logged run: {mlflow.active_run().info.run_id}")
        print(f"MAP@5 = {metrics['map@5']:.3f} | p95 latency =
{metrics['p95_ms']}ms")


if __name__ == '__main__':
    main()


# ---- Run the sweep ----
# for model in BAAI/bge-base-en-v1.5 sentence-transformers/all-MiniLM-L6-v2;
do
#   for size in 300 500 800; do
```

```
#     python experiments/track_retrieval_run.py --model $model --chunk-size
$size
#   done
# done
# Then: mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db
# Open http://localhost:5000, select all runs, click Compare.
```

python — Programmatic run query - picking the winner from a sweep

```python
# experiments/find_best_run.py
# Query MLflow to find the best run matching multiple criteria.
# Used by CI or a promotion script to auto-select which model to register
(Section 4.5).

import mlflow
import pandas as pd

mlflow.set_tracking_uri('http://localhost:5000')

# ---- Query 1: all runs in the experiment, sorted by quality ----
df = mlflow.search_runs(experiment_names=['askmynotes-retrieval'])
cols = ['run_id', 'tags.mlflow.runName',
        'params.embedding_model', 'params.chunk_size',
        'metrics.map_at_5', 'metrics.p95_latency_ms']
print(df[cols].sort_values('metrics.map_at_5', ascending=False).head(10))


# ---- Query 2: filter by params + metric constraint ----
# Find the best MAP@5 among runs whose p95 latency is under 200ms
candidates = mlflow.search_runs(
    experiment_names=['askmynotes-retrieval'],
    filter_string="metrics.p95_latency_ms < 200 "
                  "and params.chunk_overlap = '50'",
    order_by=['metrics.map_at_5 DESC'],
    max_results=5,
)

if candidates.empty:
    print('No runs meet criteria - loosen the filters or run more
experiments')
else:
    winner = candidates.iloc[0]
    print(f'Best run: {winner.run_id}')
    print(f"  Model: {winner['params.embedding_model']}")
    print(f"  Chunk size: {winner['params.chunk_size']}")
```

```
    print(f"  MAP@5: {winner['metrics.map_at_5']:.3f}")
    print(f"  P95 latency: {winner['metrics.p95_latency_ms']}ms")

    # This run_id feeds into Section 4.5's mlflow.register_model call:
    #   mlflow.register_model(f'runs:/{winner.run_id}/model',
'AskMyNotesRetriever')


# ---- Query 3: reproduce an old run ----
old_run = mlflow.get_run('abc123def456...')
print('Params:', dict(old_run.data.params))
print('Metrics:', dict(old_run.data.metrics))
print('Git SHA:', old_run.data.tags.get('git_sha'))
# git checkout <sha>; rerun script with same params -> reproducible
experiment
```

Student

Time to get concrete. Walk me through MLflow.

Claude • AI Assistant

MLflow is the most-adopted open-source experiment-tracking tool in machine learning. It's a Python library + local UI + optional server that logs every training run's inputs (params) and outputs (metrics, artifacts) to a queryable store, then lets you compare runs, promote the best one, and reproduce any old run months later.

This subsection is Chapter 4's high-emphasis section because MLflow (or a direct competitor - W&B, Neptune, Comet) is the tool you'll reach for on your very first day of any ML engineering job. Everything in Chapter 5 (pipeline automation, model deployment, drift-triggered retraining) assumes you have MLflow-style tracking running.

By the end of this subsection, AskMyNotes has a real MLflow tracking server logging every embedding-model experiment, with a comparison table showing which config wins on retrieval quality.

THE 60-SECOND INSTALLATION

Student

How do I get started?

Claude • AI Assistant

EXAMPLE CARD

Three commands:

bash — Code Example

```bash
pip install mlflow
mkdir -p mlruns
mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db
```

Open http://localhost:5000. You now have a working MLflow server. Every subsequent `mlflow.log_*` call in Python code lands in the SQLite-backed store; the UI shows runs, comparisons, and artifact previews.

YOUR FIRST TRACKED EXPERIMENT

Student

Show me the first tracked run.

Claude • AI Assistant

EXAMPLE CARD

A minimal tracked training run for AskMyNotes' chunker + embedder combo:

python — Code Example

```python
# experiments/track_embedding_run.py
import mlflow
from askmynotes.chunker import chunk_text
from askmynotes.embedder import embed_batch
from askmynotes.eval import evaluate_retrieval
from askmynotes.data import load_golden_qa

mlflow.set_tracking_uri('http://localhost:5000')
mlflow.set_experiment('askmynotes-retrieval')

with mlflow.start_run(run_name='bge-base-cs500-ov50'):
```

```
    # 1. Log parameters (fixed inputs)
    mlflow.log_params({
        'embedding_model': 'BAAI/bge-base-en-v1.5',
        'chunk_size': 500,
        'chunk_overlap': 50,
        'top_k': 5,
    })

    # 2. Do the work
    docs = load_golden_qa()
    chunks = [chunk_text(d.text, size=500, overlap=50) for d in docs]
    embeddings = embed_batch(sum(chunks, []), model='BAAI/bge-base-en-v1.5')
    metrics = evaluate_retrieval(embeddings, top_k=5)

    # 3. Log metrics (measured outputs)
    mlflow.log_metrics({
        'map_at_5': metrics['map@5'],
        'mrr': metrics['mrr'],
        'recall_at_10': metrics['recall@10'],
        'p95_latency_ms': metrics['p95_ms'],
    })

    # 4. Log artifacts (files) - plots, sample outputs, configs
    metrics['confusion_plot'].savefig('confusion.png')
    mlflow.log_artifact('confusion.png')
    mlflow.log_dict(metrics['sample_predictions'],
'sample_predictions.json')
```

Run this and open http://localhost:5000. You'll see the run under the `askmynotes-retrieval` experiment, with all params, metrics, and artifacts visible.

PARAMS VS METRICS VS ARTIFACTS

Student

What goes in each of the three log types?

Claude · AI Assistant

CONCEPT CARD

Clear rules:

Parameters = **fixed inputs you set before the run**. They don't change during the run.

Examples: `learning_rate=0.001` , `embedding_model='bge-base'` , `chunk_size=500` , `random_seed=42` . If you'd write it in a config file, it's a param.

Logged with `mlflow.log_param(name, value)` or `mlflow.log_params({dict})` .

Metrics = **measured outputs**. Can be single values or time series across steps/epochs.

Examples: `val_loss` per epoch, `map@5` at the end of training, `retrieval_p95_ms` measured on eval set. Logged with `mlflow.log_metric(name, value)` or with a `step=` argument to build a curve.

Artifacts = **files produced by the run**. Trained model files, plots, sample predictions, confusion matrices, config snapshots, HTML reports.

Logged with `mlflow.log_artifact(path)` for one file, `mlflow.log_artifacts(dir)` for a directory, `mlflow.log_dict(d, name.json)` for an in-memory dict, `mlflow.log_figure(fig, name.png)` for a matplotlib figure.

Rule of thumb: if you'd want to see it in the UI as a NUMBER for comparison across runs, it's a metric. If you'd want to see it as a FILE you can download, it's an artifact. If you'd want to filter/group runs by it, it's a parameter.

RUNS, EXPERIMENTS, AND ORGANIZATION

Student

How is my tracking data organized?

Claude • AI Assistant

CONCEPT CARD

Two-level hierarchy:

Experiment = a grouping of related runs, usually corresponding to a project or a research question. Examples: 'askmynotes-retrieval', 'askmynotes-reranker', 'chunk-size-sweep'. Create/select with `mlflow.set_experiment('name')` .

Run = one training or evaluation execution within an experiment. Each run has an auto-generated ID plus optional `run_name` . Create with `mlflow.start_run(run_name='...')` . Every run gets its own params, metrics, and artifacts.

Nested runs (advanced): a run can spawn child runs (`with mlflow.start_run(nested=True):`). Useful for hyperparameter searches - parent run summarizes the sweep, each trial is a child.

Tag runs for extra filtering: `mlflow.set_tag('git_sha', 'abc123')` , `mlflow.set_tag('env', 'laptop')` . Tags don't participate in the params/metrics UI but are queryable.

AUTOLOGGING - THE ONE-LINER PATH

Student

Do I have to write all these log calls by hand?

Claude • AI Assistant

CONCEPT CARD

Not for standard frameworks. **Autologging** hooks into scikit-learn, PyTorch Lightning, Hugging Face Transformers, XGBoost, LightGBM, Keras, TensorFlow, and others - logs everything automatically:

python — Code Example

```python
import mlflow
import mlflow.sklearn

mlflow.sklearn.autolog()                # <-- the one line

# Everything below is autologged: hyperparams, training metrics, model file, signature
with mlflow.start_run():
    model = RandomForestClassifier(n_estimators=200, max_depth=10)
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
```

For Hugging Face:

python — Code Example

```python
import mlflow
mlflow.transformers.autolog()

# training_args, per-step loss, tokenizer, model all logged
trainer = Trainer(...)
trainer.train()
```

Rule: **start with autologging** for any framework it supports; add manual `log_metric` / `log_artifact` calls for domain-specific things (retrieval quality, latency, sample predictions) autologging doesn't know about.

COMPARING RUNS

Student

What does the UI actually give me?

Claude • AI Assistant

CONCEPT CARD

The MLflow UI's core value is **run comparison**. Select multiple runs (checkbox in the runs list); click 'Compare'. You get:

- **Params diff table**: which params differ across runs, highlighted.
- **Metrics side-by-side**: for scalar metrics, a bar chart per metric per run. For per-step metrics, overlaid curves.
- **Parallel coordinates plot**: for a hyperparameter sweep, this shows how each param + each metric are correlated across all runs. Instantly reveals which knobs matter.
- **Scatter plot**: pick any two axes; see the tradeoff. Common use: 'retrieval quality vs latency' - which model is Pareto-optimal?
- **Artifacts side-by-side**: preview images (confusion matrices, learning curves) across runs.

The compare view is where 'we tried a bunch of things' becomes 'here's the plot showing bge-base beats MiniLM by 8 points on MAP@5 but is 3x slower - here's why we chose bge-base with smaller top-K'.

QUERYING RUNS PROGRAMMATICALLY

Student

Can I query runs from Python?

Claude • AI Assistant

CONCEPT CARD

Yes - the `mlflow.search_runs` API:

python — Code Example

```python
import mlflow

# All runs in one experiment as a Pandas DataFrame
df = mlflow.search_runs(experiment_names=['askmynotes-retrieval'])
df[['run_id', 'params.embedding_model', 'metrics.map_at_5',
'metrics.p95_latency_ms']]\
    .sort_values('metrics.map_at_5', ascending=False)\
    .head(5)

# Filter by params/metrics with a SQL-like syntax
best = mlflow.search_runs(
    experiment_names=['askmynotes-retrieval'],
    filter_string="params.embedding_model = 'BAAI/bge-base-en-v1.5' "
                  "and metrics.map_at_5 > 0.75",
    order_by=['metrics.map_at_5 DESC'],
    max_results=1,
)
```

This becomes the 'pick the best run and promote it to the Model Registry' step (Section 4.5). Also useful for weekly summary emails, per-team dashboards, or driving retraining decisions from a scheduled script.

REPRODUCING A RUN MONTHS LATER

Student

How does MLflow help me reproduce a run?

Claude • AI Assistant

CONCEPT CARD

Two mechanisms:

1. Everything logged is queryable: `mlflow.get_run(run_id)` returns the exact params, metrics, artifacts, and tags.

Combined with `mlflow.set_tag('git_sha', <sha>)` at run time, you can `git checkout <sha>` and re-run with the identical params. Best practice: **always log the git SHA and requirements.txt or `pip freeze` as tags/artifacts**.

2. MLflow Projects (`MLproject` file): declares dependencies, entry points, and default params.

`mlflow run <uri> -P param=value` builds a reproducible env and executes the entry point. Any teammate (or CI) can run the same experiment with `mlflow run https://github.com/you/askmynotes -P embedding_model=bge-base` .

This is how CI pipelines trigger training runs.

Reproducibility of the model file itself is a stronger claim - depends on deterministic training (fixed random seeds, single-threaded ops, matching hardware). MLflow doesn't solve that; it just makes the inputs to reproducibility fully knowable.

LOCAL VS REMOTE MLFLOW SERVERS

Student

Do I always run MLflow locally?

Claude • AI Assistant

CONCEPT CARD

Three deployment patterns:

Local file store (default): runs saved to `./mlruns/` . Great for solo work. No sharing.

Local server + SQLite: `mlflow ui --backend-store-uri sqlite:///mlflow.db` - queryable via search API. Still solo, but faster and supports the Model Registry.

Remote server (team): dedicated MLflow server backed by Postgres for metadata and S3/GCS for artifacts. Team members set `MLFLOW_TRACKING_URI=https://mlflow.example.com` in their env; runs land in the shared store; everyone sees everything.

Also standard for CI-driven training runs.

For AskMyNotes as a learning project: local server + SQLite is exactly right. When you graduate to a real team, ask what MLflow URI to point at - the client code is unchanged.

ANALOGY

MLflow is **Git for ML experiments**. Git commits your code with a message and lets you compare/rewind.

MLflow 'commits' every training run with params + metrics + artifacts and lets you compare/rewind. Where Git tracks source; MLflow tracks the outputs of running that source with specific inputs on specific data.

Both are indispensable; both are cheap to install; both make future-you enormously happier than past-you.

REAL WORLD APPLICATION

A common story: a data scientist trained ~50 models over three months in Jupyter notebooks, picked one, deployed it. Six months later the model degraded.

Nobody could reproduce the training - the original notebook had been edited many times, the data snapshot was gone, hyperparams were remembered vaguely. The company had to retrain from scratch, taking two weeks.

After switching to MLflow: the same scenario becomes 'pull run X's params and code SHA, retrain, done in an afternoon'. **Tracking is cheap; not tracking is expensive**.

WARNING

Don't log giant artifacts (>100MB) casually. Model checkpoints for large models can bloat your MLflow store fast.

Log a summary (metrics + a small sample of predictions + a plot); log the full checkpoint only for runs you're likely to promote (or use MLflow's registry integration which stores model files once, referenced by version).

COMMON PITFALL / MISTAKE

Logging metrics without params. 'This run got 0.82' - great, but what made it 0.82?

What was the learning rate? The dataset version?

You cannot reproduce or learn from a metric divorced from its inputs. **Log params BEFORE the work starts; log metrics as they arrive; log artifacts at the end.** In that order, always.

EXAM TIP CARD

Interview question: 'Walk me through how you'd track ML experiments.' Confident answer: 'I use MLflow. Every training run is wrapped in `with mlflow.start_run():` inside a named experiment.

I log params before training starts (hyperparams, model choice, dataset version), metrics during and after (loss curves, eval metrics, latency), artifacts at the end (model file, confusion matrix, sample predictions), and set tags for git SHA and env. The MLflow UI's compare view surfaces winners; `search_runs` gives programmatic filtering.

For sklearn / HF / Lightning I enable autologging so I only add domain-specific logs manually.'

PRACTICE QUIZ

You run 20 embedding-model experiments. Three months later you want to promote the one with the best `map@5` that also had `p95_latency_ms < 200`. What's the fastest way?

A) Grep through Jupyter notebooks

B) `mlflow.search_runs(filter_string="metrics.p95_latency_ms < 200", order_by=['metrics.map_at_5 DESC'], max_results=1)`

C) Ask your teammates

D) Rerun all 20 experiments

▼ Reveal Correct Answer

Answer: B. The whole point of tracking is that this becomes a one-line query. A, C, D are what teams DO without tracking - painfully.

EXERCISE

On AskMyNotes: (a) start a local MLflow server with `mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db`, (b) create `experiments/track_run.py` that logs params + metrics + one artifact for a chunker + embedder combo evaluated on a small Q&A set, (c) run it 5 times varying `chunk_size` in `[200, 400, 600, 800, 1000]`, (d) open the UI, select all 5 runs, click Compare, and identify the winner, (e) programmatically query the best run with `mlflow.search_runs` and print the winning chunk_size.

AI DEEP DIVE PROMPT

Ask an AI: *'Write MLflow tracking code for a hyperparameter sweep over embedding models (MiniLM, bge-base, text-embedding-3-small) x chunk sizes ([300, 500, 800]) x overlaps ([0, 50, 100]) - 27 runs total. Include autologging where possible, manual params for the sweep dimensions, metrics for map@5 and latency, and artifacts for a per-run confusion matrix.

Use nested runs so the sweep has a parent run summarizing the best trial.'*

SUMMARY CARD

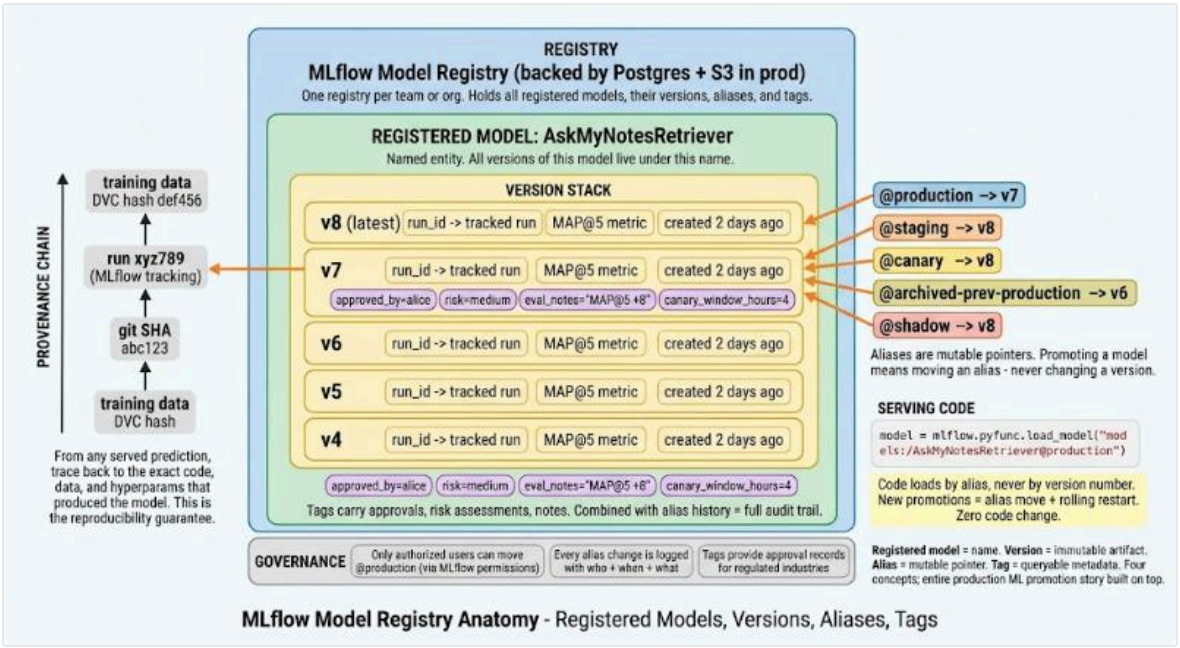
MLflow tracking is the tracking layer above Git for ML experiments. Wrap runs with `with mlflow.start_run(run_name=...):` inside a named `set_experiment()`.

Log **params** (fixed inputs) with `log_params({dict})`, **metrics** (measured outputs, optionally per-step) with `log_metric(name, value, step=...)`, **artifacts** (files) with `log_artifact(path)`. Use **autologging** (`mlflow.sklearn.autolog()`,

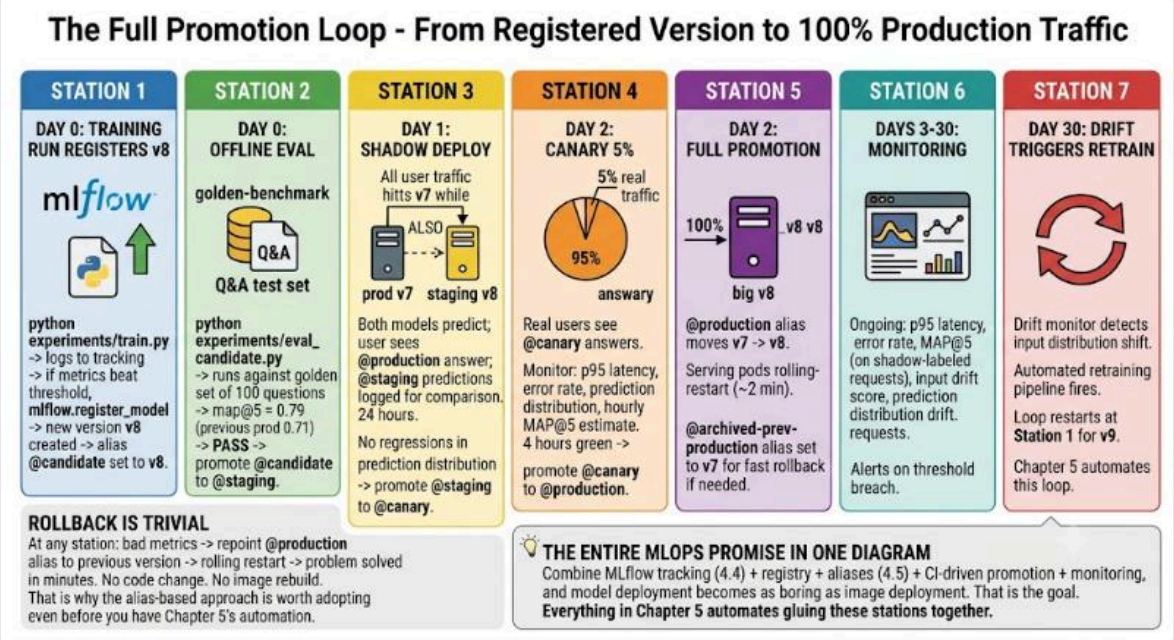
> `mlflow.transformers.autolog()`) for standard frameworks; add manual calls for domain-specific things.
>
> The UI's **Compare** view + **`mlflow.search_runs`** are how you find winners. **Always log the git SHA as a tag** and log `pip freeze` as an artifact for reproducibility.
>
> Local file store for solo work; SQLite-backed local server for the registry; remote server + Postgres + S3 for teams. Section 4.5 uses MLflow's Model Registry to promote the winners of these runs through Staging -> Production.

MLFLOW MODEL REGISTRY - THE BRIDGE FROM TRAINING TO PRODUCTION

Claude • AI Assistant



MLflow Model Registry Anatomy - Registered Models, Versions, Aliases, Tags



The Full Promotion Loop - From Registered Version to 100% Production Traffic

python — Complete promotion script for AskMyNotes - CI would run this

```python
# experiments/promote_best_model.py
# Query MLflow, register the winner, run offline eval, promote through aliases.
# Intended to be run from CI after a training job finishes.

import argparse
import sys
from datetime import datetime, timezone

import mlflow
from mlflow import MlflowClient

REGISTERED_MODEL = 'AskMyNotesRetriever'
EXPERIMENT_NAME = 'askmynotes-retrieval'
LATENCY_BUDGET_MS = 200
MIN_MAP_AT_5 = 0.75


def find_best_run(client: MlflowClient) -> str:
    runs = mlflow.search_runs(
        experiment_names=[EXPERIMENT_NAME],
        filter_string=f"metrics.p95_latency_ms < {LATENCY_BUDGET_MS} "
                      f"and metrics.map_at_5 >= {MIN_MAP_AT_5}",
        order_by=['metrics.map_at_5 DESC'],
        max_results=1,
    )
    if runs.empty:
```

```
        print(f'No runs met criteria (map_at_5>={MIN_MAP_AT_5},
p95<{LATENCY_BUDGET_MS}ms)')
        sys.exit(1)
    return runs.iloc[0]


def register(client: MlflowClient, run_id: str) -> int:
    mv = mlflow.register_model(
        model_uri=f'runs:/{run_id}/model',
        name=REGISTERED_MODEL,
    )
    print(f'Registered {REGISTERED_MODEL} v{mv.version} from run {run_id}')
    return int(mv.version)


def tag_version(client: MlflowClient, version: int, run: 'pd.Series') ->
None:
    now = datetime.now(timezone.utc).isoformat()
    tags = {
        'registered_at': now,
        'registered_by': 'ci-bot',
        'source_run_id': run.run_id,
        'git_sha': run.get('tags.git_sha', 'unknown'),
        'map_at_5': f"{run['metrics.map_at_5']:.4f}",
        'p95_latency_ms': f"{run['metrics.p95_latency_ms']:.1f}",
        'risk_level': 'medium',
        'eval_status': 'candidate',
    }
    for k, v in tags.items():
        client.set_model_version_tag(REGISTERED_MODEL, version, k, str(v))


def promote_alias(client: MlflowClient, version: int, alias: str) -> None:
    client.set_registered_model_alias(REGISTERED_MODEL, alias, version)
    print(f'{REGISTERED_MODEL}@{alias} -> v{version}')


def offline_eval_passes(client: MlflowClient, version: int) -> bool:
    # In practice: load the model, run against a held-out golden set, check
metrics.
    # For this template: fetch the training metric and re-check thresholds.
    mv = client.get_model_version(REGISTERED_MODEL, str(version))
    tags = {t.key: t.value for t in mv.tags} if False else {
        t: v for t, v in client.get_model_version(REGISTERED_MODEL,
        str(version)).tags.items()
    }
```

```
    map5 = float(tags.get('map_at_5', 0))
    p95 = float(tags.get('p95_latency_ms', 999))
    passed = map5 >= MIN_MAP_AT_5 and p95 < LATENCY_BUDGET_MS
    print(f'Offline eval: MAP@5={map5:.3f} (>= {MIN_MAP_AT_5}?), '
          f'p95={p95:.1f}ms (< {LATENCY_BUDGET_MS}?) -> {"PASS" if passed
else "FAIL"}')
    return passed


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--tracking-uri', default='http://localhost:5000')
    ap.add_argument('--target-alias', default='staging',
                    choices=['candidate', 'staging', 'canary', 'production'])
    args = ap.parse_args()

    mlflow.set_tracking_uri(args.tracking_uri)
    client = MlflowClient()

    best = find_best_run(client)
    version = register(client, best.run_id)
    tag_version(client, version, best)
    promote_alias(client, version, 'candidate')

    if args.target_alias == 'candidate':
        return                                  # done: just registered as
candidate

    if not offline_eval_passes(client, version):
        print('Offline eval failed - remaining at @candidate')
        sys.exit(1)

    promote_alias(client, version, args.target_alias)
    client.set_model_version_tag(
        REGISTERED_MODEL, version, 'eval_status', args.target_alias,
    )
    print(f'Promotion complete: {REGISTERED_MODEL} v{version} @
{args.target_alias}')


if __name__ == '__main__':
    main()


# ---- Wire into CI (add to .github/workflows/train.yml) ----
#   - name: Register + promote to staging
```

```
#     env:
#       MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }}
#     run: python experiments/promote_best_model.py --target-alias staging
```

python — Serving code that loads by alias - the whole production integration

```python
# askmynotes/serving.py
# The whole production integration with the MLflow registry: 20 lines.

import os
from functools import lru_cache

import mlflow
import mlflow.pyfunc
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

MODEL_NAME = os.environ.get('MODEL_NAME', 'AskMyNotesRetriever')
MODEL_ALIAS = os.environ.get('MODEL_ALIAS', 'production')
MLFLOW_URI = os.environ.get('MLFLOW_TRACKING_URI', 'http://localhost:5000')

mlflow.set_tracking_uri(MLFLOW_URI)


@lru_cache(maxsize=1)
def load_model():
    """Load once at first request; cached for the process lifetime."""
    uri = f'models:/{MODEL_NAME}@{MODEL_ALIAS}'
    print(f'Loading {uri}')
    return mlflow.pyfunc.load_model(uri)


app = FastAPI()


class RerankRequest(BaseModel):
    query: str
    candidates: list[str]


class RerankResponse(BaseModel):
    scores: list[float]
    model_alias: str
```

```
@app.get('/health')
def health():
    return {'status': 'ok', 'model_alias': MODEL_ALIAS}


@app.post('/rerank', response_model=RerankResponse)
def rerank(req: RerankRequest):
    if not req.candidates:
        raise HTTPException(400, 'candidates cannot be empty')
    model = load_model()
    scores = model.predict({'query': req.query, 'candidates':
req.candidates})
    return RerankResponse(scores=list(scores), model_alias=MODEL_ALIAS)


# ---- Run locally ----
# uvicorn askmynotes.serving:app --host 0.0.0.0 --port 8080

# ---- Promote a new version WITHOUT touching this code ----
# python -c "from mlflow import MlflowClient; \
#            MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 8)"
# kubectl rollout restart deployment/askmynotes-serving       # or equivalent
# The next request loads v8 via the @production alias. Zero code change.
```

Student

MLflow tracking captures every run. But how does one of those runs become 'the model that's serving users'?

Claude · AI Assistant

The **MLflow Model Registry** is the bridge. It's a named, versioned store of trained models sitting on top of your tracking server - the equivalent of a Docker registry, but for ML models.

This subsection covers how to register a model from a tracked run, how to promote versions through the Staging / Production ladder using **aliases** (the modern approach; **stages** are the older deprecated approach), how to load a model by alias in your serving code (so the code never hardcodes a version), and how the registry integrates with CI/CD to

make model deploys as automated as image deploys. Together with Section 4.4, this closes MLflow's dev-to-prod loop for AskMyNotes.

WHY A MODEL REGISTRY

Student

Why not just save models as files in Git or S3?

Claude • AI Assistant

CONCEPT CARD

Four reasons a registry beats loose files:

1. Named handles instead of paths. Serving code loads `models:/AskMyNotesRetriever@production` - a symbolic name. It doesn't care whether that resolves to v7 or v42. The person deploying a new model doesn't need to touch the serving code; they change the alias.

2. Full provenance for every version. Each registered version links back to the tracked run that produced it - so you can see params, metrics, the git SHA, the training dataset version.

From a production model file, you can trace all the way back to 'this was trained on Oct 12 by run xyz on data version abc'.

3. Approval workflows built in. Only authorized users can change the `@production` alias. Every alias change is logged with who + when. This is the ML equivalent of branch protection.

4. Signature and framework metadata. The registry stores the model's input/output schema (signature) and its framework flavor (sklearn, PyTorch, Transformers, ...) so serving code doesn't have to know these details ahead of time.

Loose files in S3 give you none of this. A registry is a small amount of infrastructure that removes an enormous class of production ML problems.

REGISTERING A MODEL

Student

How do I get a model INTO the registry?

Claude · AI Assistant

EXAMPLE CARD

Two-step process. In your training script (extending 4.4's example):

python — Code Example

```python
import mlflow
from mlflow.models import infer_signature
import mlflow.sklearn

with mlflow.start_run(run_name='bge-base-cs500-ov50') as run:
    mlflow.log_params({...})

    # ... do the training ...
    model = train_retriever(...)

    mlflow.log_metrics({'map_at_5': 0.782, 'p95_latency_ms': 182})

    # Save model with signature - inference schema is stored alongside the weights
    signature = infer_signature(sample_input, sample_output)
    mlflow.sklearn.log_model(
        sk_model=model,
        artifact_path='model',
        signature=signature,
        input_example=sample_input,
    )

    # THIS is the registration - creates version 1 (or v2, v3, ... on subsequent runs)
    mlflow.register_model(
        model_uri=f'runs:/{run.info.run_id}/model',
        name='AskMyNotesRetriever',
    )
```

Or, after a run: use the search API to find the winner and register it separately:

python — Code Example

```python
best = mlflow.search_runs(
    experiment_names=['askmynotes-retrieval'],
    filter_string="metrics.p95_latency_ms < 200",
    order_by=['metrics.map_at_5 DESC'],
    max_results=1,
```

```
).iloc[0]

mlflow.register_model(
    model_uri=f'runs:/{best.run_id}/model',
    name='AskMyNotesRetriever',
)
```

Each `register_model` call creates a new **version** under the named registered model. Version numbers are auto-assigned and immutable - v3 is always v3, forever.

ALIASES - THE MODERN PROMOTION PRIMITIVE

Student

How do I mark a version as 'the production model'?

Claude • AI Assistant

CONCEPT CARD

With an **alias**. Aliases are mutable pointers from a name to a specific version - `@production` , `@staging` , `@challenger` , `@shadow` , `@champion` - any label your team uses.

They replaced the older fixed-vocabulary **stages** (None / Staging / Production / Archived), which MLflow now considers deprecated for new work.

python — Code Example

```python
from mlflow import MlflowClient

client = MlflowClient()

# Promote v7 to production - moves the @production pointer to v7
client.set_registered_model_alias(
    name='AskMyNotesRetriever',
    alias='production',
    version=7,
)

# Retire the previous production version to @archived
client.set_registered_model_alias(
    name='AskMyNotesRetriever',
    alias='archived-prev-production',
```

```
    version=6,
)

# Delete an alias (e.g., end a canary window)
client.delete_registered_model_alias(
    name='AskMyNotesRetriever',
    alias='canary',
)
```

Why aliases beat stages: any number of aliases (not just four), self-documenting (`@challenger` , `@shadow` , `@eu-only`), and they compose with tags for governance. A team can invent whatever promotion vocabulary matches their process.

The old stage-based API (`transition_model_version_stage(stage='Production')`) still works and much production code uses it - be able to read it in inherited codebases - but prefer aliases in new code.

TAGS FOR GOVERNANCE

Student

What about approval and audit?

Claude · AI Assistant

CONCEPT CARD

Tags are key-value metadata on registered models or specific versions. Use them for approval records, risk assessments, or free-form notes:

python — Code Example

```
client.set_model_version_tag(
    name='AskMyNotesRetriever',
    version=7,
    key='approved_by',
    value='alice@company.com',
)
client.set_model_version_tag(
    name='AskMyNotesRetriever',
    version=7,
    key='risk_level',
    value='medium',
)
```

```
client.set_model_version_tag(
    name='AskMyNotesRetriever',
    version=7,
    key='eval_notes',
    value='MAP@5 +8 vs prev prod; latency within budget; passes fairness
check',
)
```

Combined with the git SHA logged in tracking and the alias history in the registry, you get a **full audit trail**: which version was promoted to production, when, by whom, based on what evaluation, from what code and data. Regulated industries (fintech, healthcare) require this level of provenance.

Non-regulated teams benefit from it too.

LOADING A MODEL BY ALIAS IN SERVING CODE

Student

How does the serving code use the registry?

Claude · AI Assistant

EXAMPLE CARD

The serving code loads by **alias**, never by version number:

python — Code Example

```python
# askmynotes/serving.py
import mlflow.pyfunc

MODEL_URI = 'models:/AskMyNotesRetriever@production'

# Loaded once at process start; served for the process lifetime
model = mlflow.pyfunc.load_model(MODEL_URI)

def rerank(query: str, candidates: list[str]) -> list[float]:
    return model.predict({'query': query, 'candidates': candidates})
```

This is the entire integration. When a new version is promoted (`@production` alias moves from v7 to v8), a rolling restart of the serving pods picks up the new model. **The application code never changes** - only the alias moves.

For hot-reload without a restart, wrap `load_model` in a periodic refresh (say, every 5 minutes) or subscribe to registry webhooks (paid MLflow variants and Databricks). Most teams find rolling restarts sufficient.

THE PROMOTION LADDER IN PRACTICE

Student

What does a real promotion sequence look like?

Claude • AI Assistant

CONCEPT CARD

Concrete flow for AskMyNotes:

Day 0 - training script runs (via CI or manually):

- Trains a new retriever
- Logs to MLflow tracking (params, metrics, artifacts)
- If eval metrics beat a threshold, calls `mlflow.register_model` -> creates v8
- Sets `@candidate` alias to v8

Day 0 - automated evaluation:

- Runs the golden Q&A benchmark on v8
- If pass: promotes `@candidate` -> `@staging`
- Tags v8 with `eval_notes` and `map_at_5=0.79`

Day 1 - shadow deploy:

- Serving cluster loads `@staging` model AS WELL AS `@production` model
- Each user request generates two predictions (both logged); user sees the `@production` answer
- After 24 hours of shadow traffic, compare prediction distributions: if no regressions, promote `@staging` -> `@canary`

Day 2 - canary:

- 5% of production traffic routed to `@canary` for its answer (not shadow anymore)
- SLO metrics + prediction distribution watched for 4 hours
- If green: promote `@canary` -> `@production` ; retire prev `@production` to `@archived-prev`

Day 2 - full production:

- `@production` alias now points at v8
- Serving pods rolling-restart to pick up the new model
- Monitoring continues for the SLO window

Day 30 - drift check triggers retraining:

- Drift monitor sees input distribution has shifted; retraining pipeline fires
- Loop restarts at Day 0 for v9

This entire flow can be scripted and driven from CI - the registry is the coordination point. Chapter 5 automates the full loop.

THE MLFLOW MODELS FORMAT

Student

What's inside a registered model?

Claude · AI Assistant

CONCEPT CARD

MLflow saves models in a standard directory layout with a `MLmodel` metadata file:

code — Code Example

```
model/
  MLmodel                    # metadata: flavor, signature, requirements
  conda.yaml                 # conda env for reproducibility
  python_env.yaml            # pyenv variant
  requirements.txt           # pip-installable deps
  model.pkl (or model.pt, ...)  # the actual weights
```

The `MLmodel` file names one or more **flavors** - loader modules that know how to reconstruct the model. `python_function` is the universal flavor - any registered model can be loaded via `mlflow.pyfunc.load_model()` regardless of framework.

`sklearn`, `pytorch`, `transformers`, `xgboost` are framework-specific flavors that expose native APIs.

The signature (input/output schema) lets serving code validate incoming requests: 'this model expects a dict with keys "query" (str) and "candidates" (list[str]) and returns list[float]'. Serving frameworks (BentoML, MLflow's built-in `mlflow models serve`, KServe) use the signature to auto-generate OpenAPI docs for the model endpoint.

SERVING A REGISTERED MODEL - THE ONE-LINER

Student

Can I just serve the model directly?

Claude • AI Assistant

EXAMPLE CARD

Yes. MLflow ships with a serving CLI:

bash — Code Example

```
mlflow models serve \
  -m models:/AskMyNotesRetriever@production \
  --host 0.0.0.0 --port 8080 \
  --env-manager virtualenv
```

This starts a Flask-based server exposing the model at `POST /invocations`. Perfect for local testing or small internal tools.

For real production traffic, use BentoML, KServe, or Triton - they add batching, autoscaling, and streaming - but they all pull models from the same MLflow registry.

The Docker image path:

bash — Code Example

```
mlflow models build-docker \
  -m models:/AskMyNotesRetriever@production \
  -n askmynotes-retriever:latest

docker run -p 8080:8080 askmynotes-retriever:latest
```

One command turns a registered model into a Docker image, ready to push to your registry and deploy through the CD pipeline from Section 4.2. **This is where MLOps and DevOps meet operationally.**

CI-DRIVEN REGISTRATION

Student

How do CI pipelines interact with the registry?

Claude • AI Assistant

EXAMPLE CARD

A typical CI job for a training pipeline (extension of Section 3.5's workflow):

yaml — Code Example

```yaml
train:
  if: contains(github.event.head_commit.message, '[train]')
  runs-on: [self-hosted, gpu]           # training needs beefier runner
  env:
    MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }}
    MLFLOW_TRACKING_TOKEN: ${{ secrets.MLFLOW_TOKEN }}
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: { python-version: '3.12', cache: 'pip' }
    - run: pip install -e .[dev]
    - name: Run training + log to MLflow
      run: python experiments/train_retriever.py --register-if-better
    - name: Promote to staging if eval passes
      run: python experiments/promote_if_eval_passes.py --alias staging
```

The `train_retriever.py` script trains, logs to MLflow, and (if metrics beat the current production model) calls `mlflow.register_model` + sets `@candidate`. The `promote_if_eval_passes.py` script queries the registry, runs the offline eval suite, and moves `@candidate` -> `@staging` on success.

From there, staged rollout to `@canary` and `@production` happens per your deployment tooling.

The result: model deployment becomes as boring as image deployment - a green pipeline that traceably moves an artifact through named environments, with rollback as easy as pointing an alias at an earlier version. That's the entire ambition of MLOps in one sentence.

ANALOGY

The Model Registry is **Docker Hub for models**, plus **GitHub branch protection for who can promote what**. Just as Docker Hub stores versioned images that clusters pull by tag, MLflow stores versioned models that services pull by alias.

And just as branch protection controls who can merge to main, registry aliases + tags control who can move `@production`. Both are small pieces of infrastructure with outsized safety returns.

REAL WORLD APPLICATION

A common progression: a team logs experiments to MLflow tracking for a month, then registers models manually when they want to promote one. After a quarter, they realize they're manually copying model paths into serving code every time.

They switch to loading by alias (`models:/X@production`). After another quarter, they wire the entire promotion pipeline into CI.

From then on, deploying a new model version is a single script command - and the audit trail (who promoted what, when, based on what metrics) exists automatically for compliance. The whole progression takes six months and pays back within weeks of each step.

WARNING

Do NOT let production serving code load a model by version number (`models:/X/7`). Every time you promote a new version, you have to redeploy the code.

Load by alias (`models:/X@production`), then a rolling restart is enough. Version-number loading is fine for offline evaluation scripts, but poisonous for the serving path.

COMMON PITFALL / MISTAKE

Sharing a single alias across environments. If both dev and production point at `@production`, a bad promotion breaks dev too.

Use environment-scoped aliases (`@dev` , `@staging` , `@production`) OR run separate registries per environment. Registries are cheap; broken production is not.

EXAM TIP CARD

Interview question: 'Walk me through how a trained model gets to production in your team.' Structured answer: 'Training script logs to MLflow, calls `register_model` to create a new version under a named registered model, sets `@candidate` alias. An automated eval job runs the golden benchmark; on pass, promotes `@candidate` to `@staging`.

Shadow deploy for a day; on no regressions, promote to `@canary`. Small % of prod traffic for hours; if SLO + quality metrics stay green, promote to `@production`.

Serving code loads `models:/<name>@production` so it never hardcodes a version - a rolling restart picks up the new model. Rollback is repointing `@production` to the previous version.'

PRACTICE QUIZ

Your serving code has `MODEL_URI = 'models:/AskMyNotesRetriever/8'` hardcoded. What's the operational cost when you promote v9?

A) Nothing - just retrain

B) You must redeploy the serving code with the new version number every time you promote

C) MLflow will auto-upgrade v8 to v9

D) The registry rejects the promotion

▼ Reveal Correct Answer

Answer: B. Hardcoding version numbers couples model promotion to code deployment. Load by alias (`@production`) instead; then a rolling restart picks up whichever version the alias points at, with no code change.

EXERCISE

On AskMyNotes: (a) modify your 4.4 tracked-run script to save the model with `mlflow.pyfunc.log_model` and a signature, (b) run 3 experiments varying `chunk_size`, (c) programmatically pick the best run and call `mlflow.register_model` -> creates AskMyNotesRetriever v1 (and v2, v3 on subsequent runs), (d) use `client.set_registered_model_alias` to set `@production` to the best version, (e) write a `load_model_by_alias.py` that loads `models:/AskMyNotesRetriever@production` and runs one inference, (f) promote a different version by moving the alias; confirm the loader picks up the new model without code change.

AI DEEP DIVE PROMPT

Ask an AI: '*Design a full MLflow-based promotion pipeline for a retrieval model: (1) training script registers new versions above a metric threshold, (2) eval script promotes @candidate to @staging, (3) shadow deploy compares @staging vs @production for one day, (4) canary rollout at 5% traffic for four hours, (5) full promotion. Include the aliases, tags, gating conditions, and the CI YAML that would drive each step.*'

SUMMARY CARD

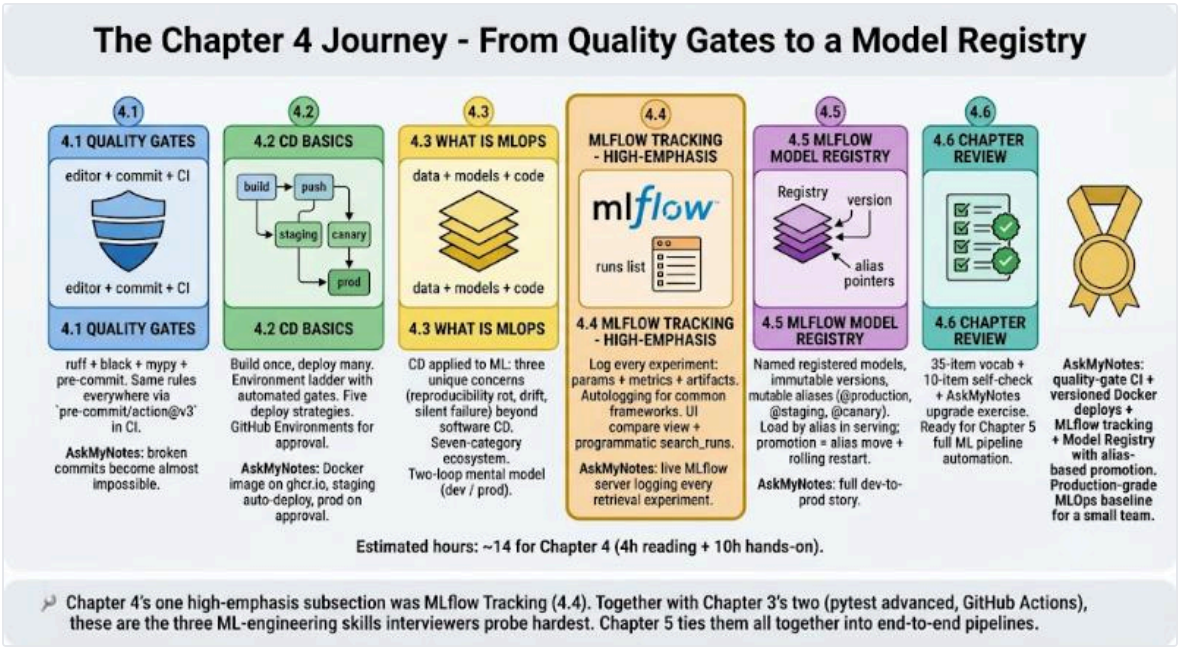
The **MLflow Model Registry** is a named, versioned store of trained models. **Registered models** are named entities (`AskMyNotesRetriever`); **versions** are auto-numbered immutable artifacts under a registered model.

Aliases (`@production` , `@staging` , `@challenger`) are mutable pointers - the modern promotion primitive, replacing the older fixed-vocabulary **stages**. **Tags** on versions capture approval + risk + notes for governance.

> Serving code loads by alias (`models:/<name>@production`) so promotion is decoupled from deploy - a rolling restart picks up the new version. **MLflow Models** are a standard format (MLmodel file + weights + requirements) with **flavors** for each framework and a **signature** for input/output schema.
>
> `mlflow models serve` and `mlflow models build-docker` turn a registered model into a running service or Docker image in one command. **CI-driven registration + eval + promotion** makes model deployment as automated as image deployment.
>
> Chapter 5 wires this into end-to-end pipelines.

CHAPTER 4 REVIEW & SUMMARY

Claude • AI Assistant



The Chapter 4 Journey - From Quality Gates to a Model Registry

bash — Chapter 4 command reference - the quality + CD + MLOps toolbox

```
# ---- Quality gates (4.1) ----
pip install ruff black mypy pre-commit bandit
pre-commit install                              # sets up .git/hooks/pre-
commit
pre-commit run --all-files                      # run all hooks on all files
pre-commit autoupdate                           # bump every hook to latest
ruff check --fix .                              # lint + auto-fix
ruff format .                                   # format (black-compatible)
```

```
black --check .                                  # CI dry-run
mypy askmynotes/                                 # type check
bandit -r askmynotes/ -c pyproject.toml          # security scan


# ---- Continuous Delivery (4.2) ----
docker build -t ghcr.io/user/askmynotes:$(git rev-parse HEAD) .
docker push ghcr.io/user/askmynotes:$(git rev-parse HEAD)
gh workflow run deploy.yml -f environment=staging
gh workflow run deploy.yml -f environment=production   # requires approval
kubectl set image deployment/askmynotes app=ghcr.io/user/askmynotes:sha-
abc123
kubectl rollout status deployment/askmynotes
kubectl rollout undo deployment/askmynotes             # rollback


# ---- MLflow tracking (4.4) ----
pip install mlflow
mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db
# In Python:
#   mlflow.set_tracking_uri('http://localhost:5000')
#   mlflow.set_experiment('askmynotes-retrieval')
#   with mlflow.start_run(run_name='...'):
#       mlflow.log_params({...})
#       mlflow.log_metrics({...})
#       mlflow.log_artifact('confusion.png')
#       mlflow.sklearn.autolog()             # for standard frameworks
# Query: mlflow.search_runs(experiment_names=[...], filter_string='...',
order_by=[...])


# ---- MLflow Model Registry (4.5) ----
# Register from a run:
#   mlflow.register_model('runs:/<id>/model', 'AskMyNotesRetriever')
# Promote via alias:
#   from mlflow import MlflowClient
#   c = MlflowClient()
#   c.set_registered_model_alias('AskMyNotesRetriever', 'production', 8)
#   c.set_model_version_tag('AskMyNotesRetriever', 8, 'approved_by', 'alice')
# Load by alias in serving:
#   model =
mlflow.pyfunc.load_model('models:/AskMyNotesRetriever@production')
# Serve as a REST endpoint:
mlflow models serve -m models:/AskMyNotesRetriever@production --host 0.0.0.0
--port 8080
# Build a Docker image from the model:
```

```
mlflow models build-docker -m models:/AskMyNotesRetriever@production -n
askmynotes:latest


# ---- Combined: CI-driven training + promotion ----
# Trigger a train workflow:
gh workflow run train.yml -f config=configs/bge_base.yaml
# Promote from CI (runs in the promote step of the workflow):
python experiments/promote_best_model.py --target-alias staging
```

CHAPTER SUMMARY

Chapter 4 extended the CI foundation (Chapter 3) into two directions: **tighter quality gates** and **model-aware CD**. **Section 4.1** wired up the three-layer defense - editor plugins, pre-commit hooks, and CI jobs - using ruff (linter/formatter, Rust-fast), black (opinionated formatter), mypy (static type checker), and pre-commit (the hook runner that makes them fire on every `git commit`).

One `.pre-commit-config.yaml` file, run in CI via `pre-commit/action@v3`, means the same rules apply locally and in the cloud with zero config drift.

Section 4.2 extended CI into Continuous Delivery - the **build-once, deploy-many** principle, the environment promotion ladder (dev -> staging -> canary -> prod), five deployment strategies (recreate / rolling / blue-green / canary / feature flag), observability as the safety net (metrics + logs + traces), SLO-driven auto-rollback, and GitHub Environments for approval gates on production deploys. Then it pivoted: **all of this transfers directly to MLOps**, with 'model + dataset' where 'image' used to be.

Section 4.3 defined MLOps as CD applied to ML systems - three uniquely ML failure modes (reproducibility rot, data drift, silent failure) beyond software CD, the seven-category MLOps stack (tracking, registry, data versioning, feature stores, pipeline orchestration, serving, drift monitoring), and the two-loop mental model (dev loop: minutes-hours; prod loop: days-weeks). **Section 4.4** (high-emphasis) got hands-on with **MLflow tracking**: experiments and runs, the params/metrics/artifacts trichotomy, autologging for standard frameworks, run comparison in the UI, programmatic queries via `search_runs`, and reproducibility via git-SHA tags plus `pip freeze` artifacts.

AskMyNotes now has a real MLflow server logging embedding-model experiments.

Section 4.5 added the **MLflow Model Registry** - the bridge from a tracked run to a served model. Registered models, immutable versions, mutable aliases (`@production`, `@staging`, `@canary` - the modern replacement for deprecated stages), tags for governance, loading by alias in serving code so promotion is a rolling restart with zero code change, and the full promotion ladder (candidate -> staging -> shadow -> canary -> production) driven from CI.

AskMyNotes now has a full dev-to-prod model story.

THE ONE-PAGE MENTAL MODEL

Software CD is 'build a known-good artifact once, promote it through a ladder of environments with automated gates, auto-rollback on SLO breach'. MLOps is exactly the same, applied to models + data + code as the artifact, with additional gates (model quality metrics, drift detection) and additional monitoring (accuracy on labeled slice, drift scores).

MLflow ties it together: tracking captures every run; the registry versions winners and exposes them via mutable aliases; serving code loads by alias so deploys are a symbol move + rolling restart. Quality gates (ruff + black + mypy + pre-commit) form the front door that catches issues before they leave the developer's machine.

WHAT'S NEXT

Chapter 5 is the capstone: **CI/CD for ML pipelines**. Everything from Chapters 1-4 comes together: data versioning with DVC, multi-step training pipelines (extract -> preprocess -> train -> validate -> register -> deploy) orchestrated with Prefect or GitHub Actions, drift-triggered retraining loops, a full end-to-end lab where AskMyNotes' retriever + reranker + generator all move through their own MLflow-registered promotion pipelines, and a consolidation subsection with interview-ready summaries of the entire Week 7 curriculum.

SELF-CHECK

Before starting Chapter 5, verify you can:

1. Explain the three-layer quality-gate defense (editor / pre-commit / CI) and configure ruff + black + mypy in `pyproject.toml`.
2. Write a `.pre-commit-config.yaml` and install its hooks with `pre-commit install`.
3. Explain build-once-deploy-many and why rebuilding for prod breaks the CD promise.
4. Describe the five deployment strategies (recreate / rolling / blue-green / canary / feature flag) with one trade-off for each.
5. Explain why silent failures are ML-specific and how drift monitoring catches them.
6. Distinguish DevOps from MLOps: name three ML-specific concerns not covered by pure software CD.
7. Set up MLflow tracking locally and log params/metrics/artifacts for a training run.
8. Query runs programmatically with `mlflow.search_runs` and pick a winner by filter + sort.
9. Register a model via `mlflow.register_model`, set an alias, and load by alias in serving code.
10. Describe the promotion ladder (candidate -> staging -> shadow -> canary -> production) and what triggers each transition.

If any are shaky, re-read the relevant subsection. Chapter 5 assumes all ten.

CHAPTER 4 EXERCISE - ASKMYNOTES GETS QUALITY GATES + A MODEL REGISTRY

Complete this before starting Chapter 5:

1. Add ruff, black, mypy, pre-commit to your dev dependencies; create the tool config sections in `pyproject.toml` .
2. Create `.pre-commit-config.yaml` with hooks for ruff (lint + format), mypy, plus basic hygiene hooks. Run `pre-commit install` .
3. Extend `.github/workflows/ci.yml` with a `quality` job using `pre-commit/action@v3` .
4. Create `Dockerfile` for AskMyNotes and a `.github/workflows/deploy.yml` that builds and pushes to `ghcr.io` on main-branch merges.
5. Add a GitHub Environment named `staging` and (optionally) `production` with a required reviewer; wire the deploy workflow to require approval for production.
6. Start a local MLflow server: `mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db` .
7. Create `experiments/track_run.py` that logs params + metrics + artifacts for a retrieval evaluation on your golden Q&A set.
8. Run 5 experiments varying `chunk_size` and 3 varying `embedding_model` (15 runs total). Compare them in the MLflow UI.
9. Write `experiments/promote_best_model.py` that finds the winner and registers it as `AskMyNotesRetriever` with the `@production` alias.
10. Modify `askmynotes/serving.py` to load `models:/AskMyNotesRetriever@production` . Promote a different version via alias-only change; confirm the loader picks it up after a restart with zero code change.

> **SUMMARY CARD**
>
> Chapter 4 built the **quality + CD + MLOps foundation** on top of Chapter 3's CI: **three-layer quality gates** (editor / pre-commit / CI) with ruff + black + mypy for shift-left code quality, **Continuous Delivery** with build-once artifacts promoted through environment ladders using rolling / canary / blue-green / feature-flag strategies, **MLOps** as CD-for-models with the added concerns of drift, silent failures, and training-serving skew, **MLflow Tracking** (high-emphasis) as the Git-for-experiments layer with params/metrics/artifacts + autologging + queryable runs, and the **MLflow Model Registry** as the bridge from tracked runs to served models via named versions and mutable aliases. AskMyNotes now has: green quality-gate CI, a deploy workflow producing versioned Docker images to ghcr.io, a live MLflow server logging every retrieval experiment, and a promotion pipeline registering + tagging + aliasing the winning model - all wireable into serving code that never hardcodes a version.

> Chapter 5 automates gluing these into full production ML pipelines.